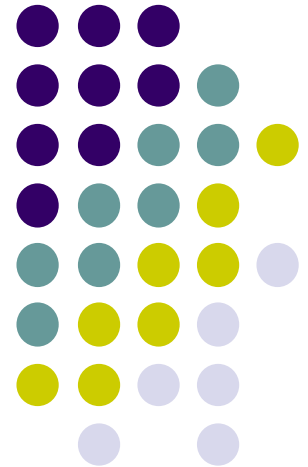
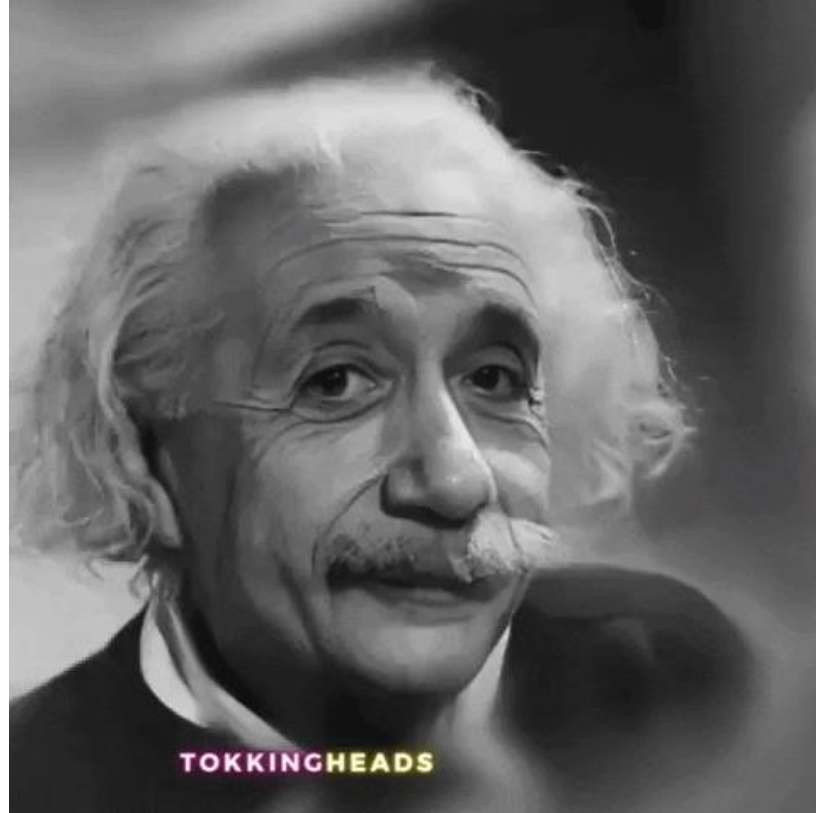
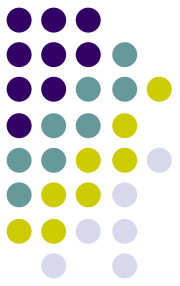


ErSE222: Generative AI: Diffusion models

May 6th, 2026



Generative



ChatGPT: How would seismic waves sound like in poetry?



*A tremor in the earth's deep chest,
A low, vibrating bass.*

*The P-wave pulses, fast and swift,
With every breath, it makes a shift.*

S-waves ripple, slower hum,

A quiver, rising, gently strum.

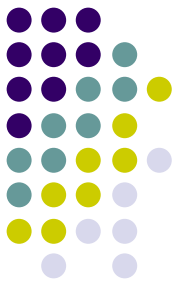
And on the surface, waves of Love

Roll like whispers from above.

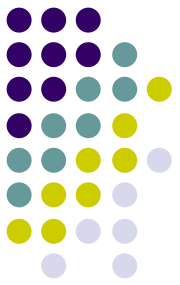
The earth sings out a rumbling song,

Both harsh and soft, short and long.

DALE-E 3: An image that depicts KAUST



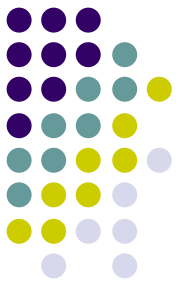
The goal of generative models



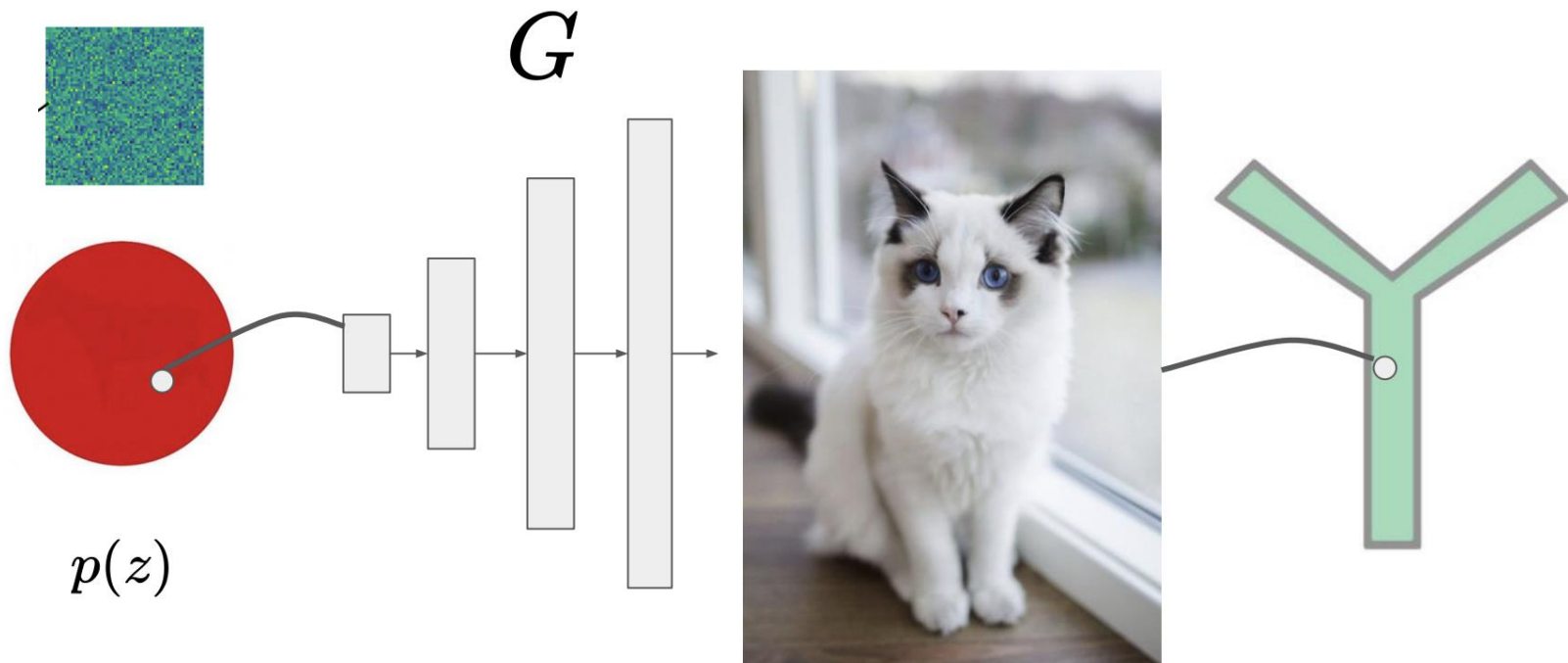
Given a distribution $p(x)$ represented by data samples

- Learn the underlying distribution $p_{\theta}(x)$ from its sample representation (we do not have knowledge of complex distributions)
- Sample $x \sim p_{\theta}(x)$ (new unseen samples) from the learned distribution
- We train to maximize the log likelihood of $p_{\theta}(x)$, or minimize divergence between $p_{\theta}(x)$ and $p_{data}(x)$

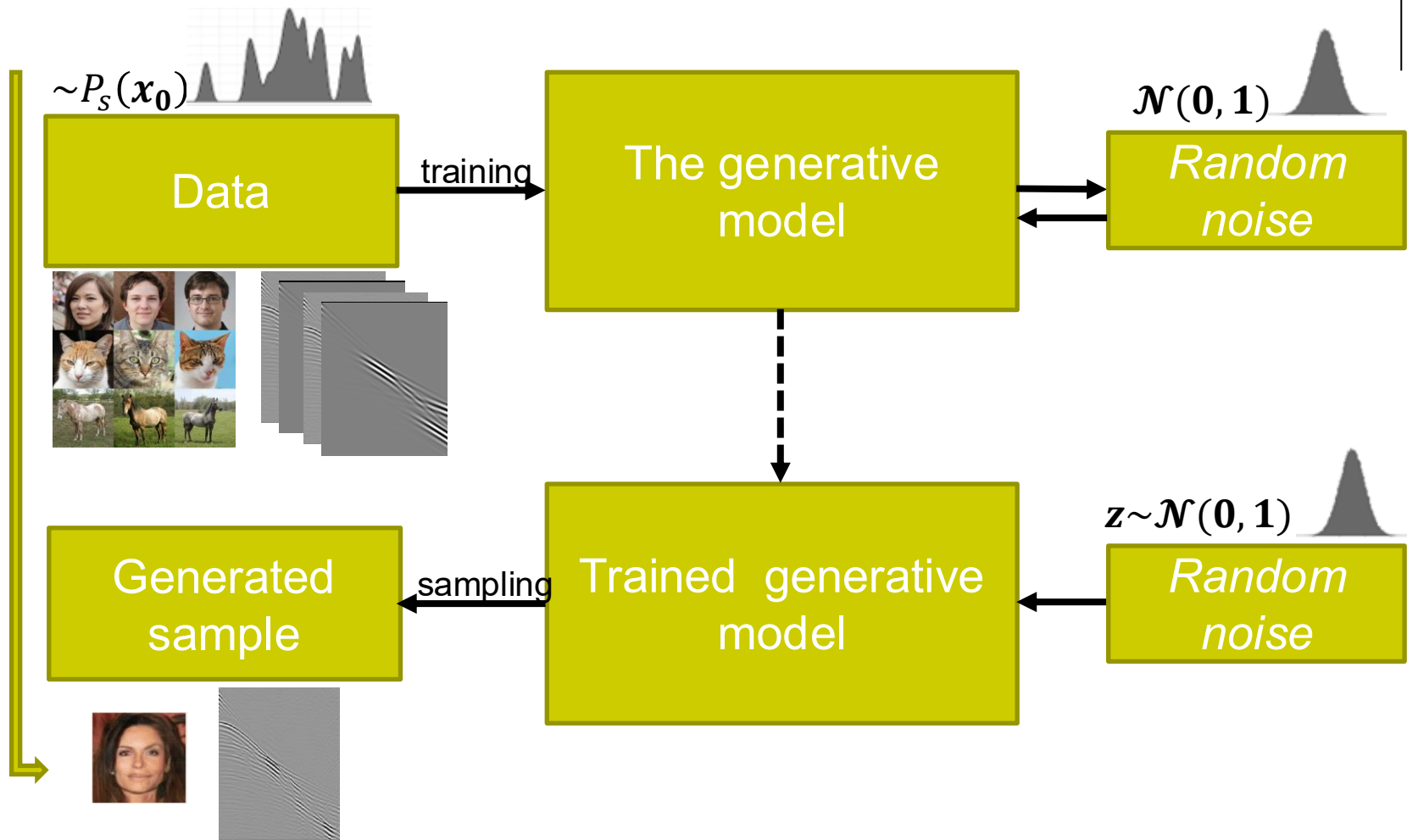
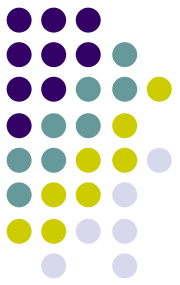
A neural network for Distributions



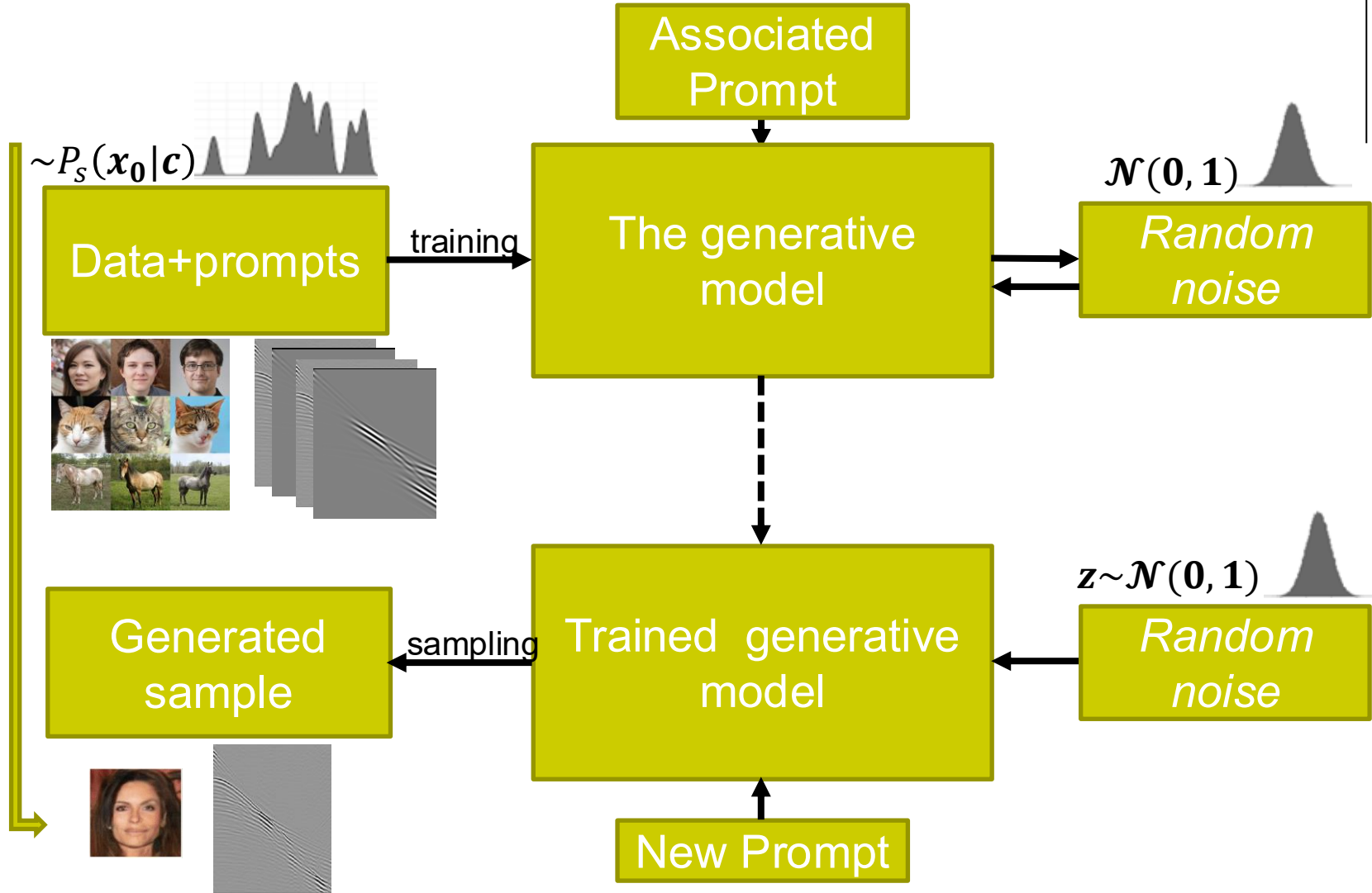
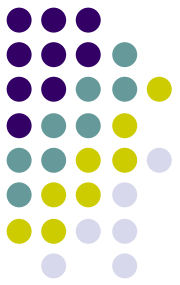
This is done by mapping the complex distribution to a simple distribution we can represent analytically



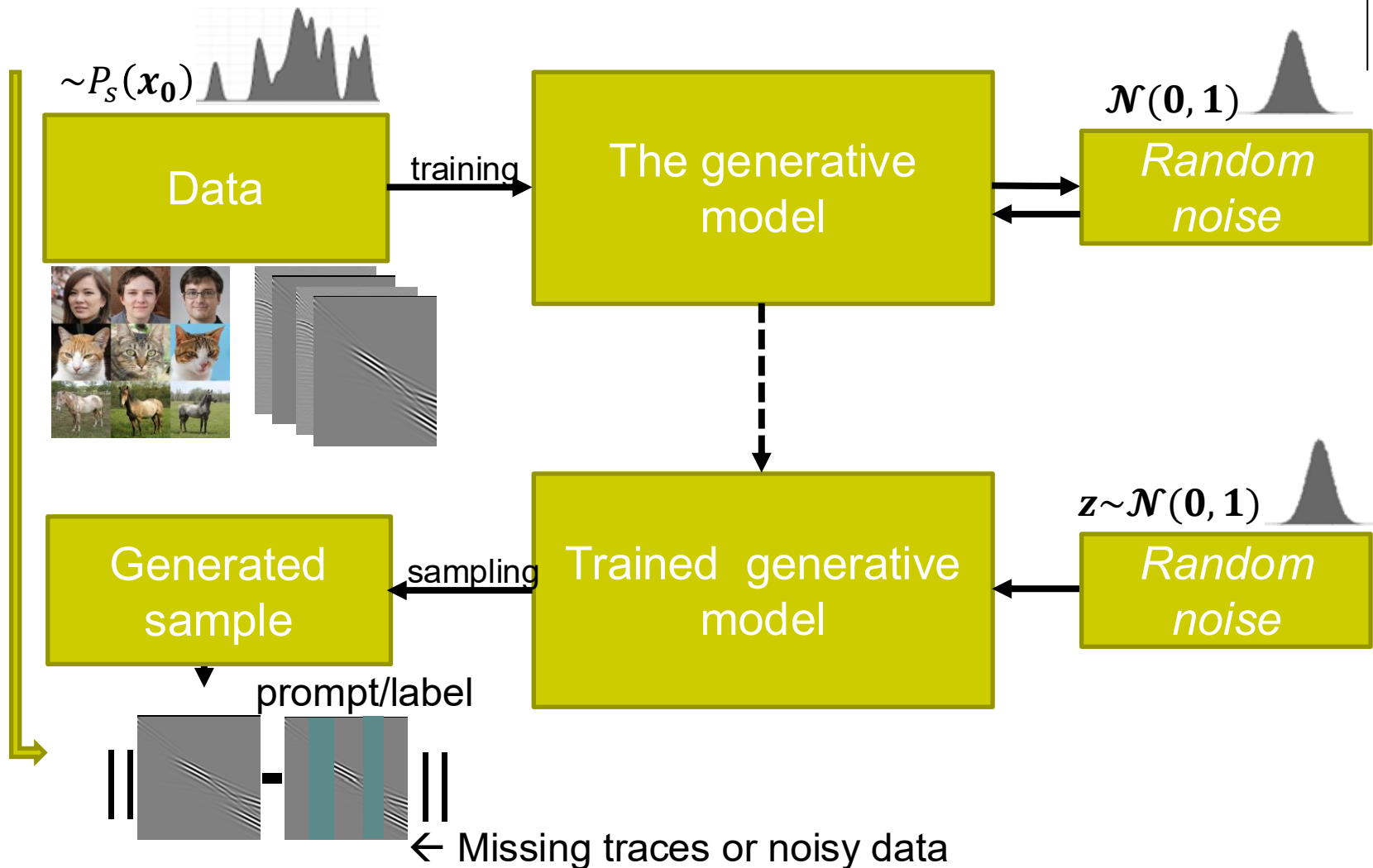
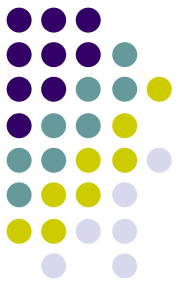
The generative model: sampling from complex distributions



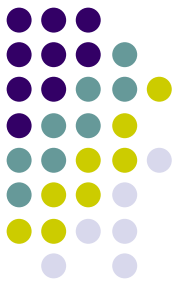
Conditional



Guidance



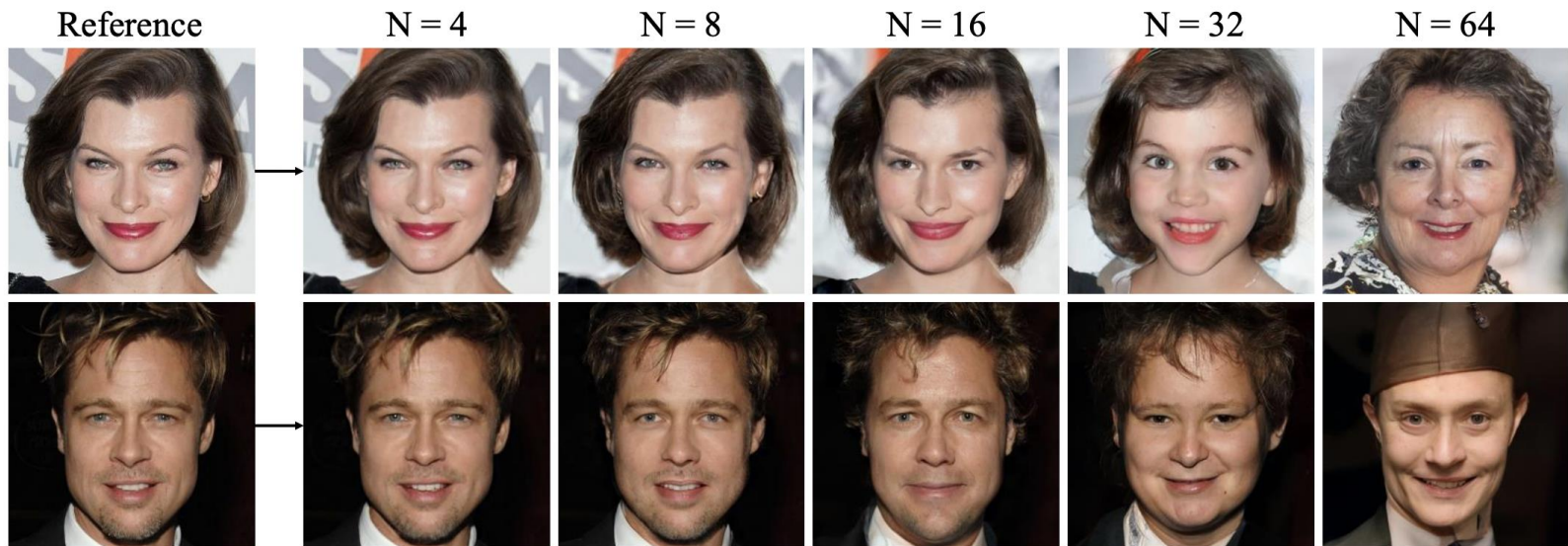
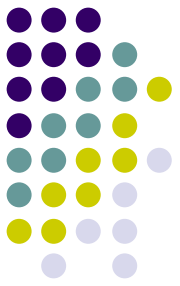
Bayesian Theorem



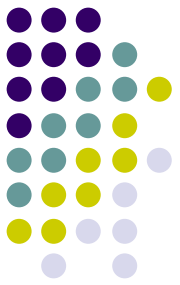
- The data as condition or guidance.

$$P(x|y) = \frac{P(y|x)P(x)}{P(y)}$$

With Guidance --- The inverse problem

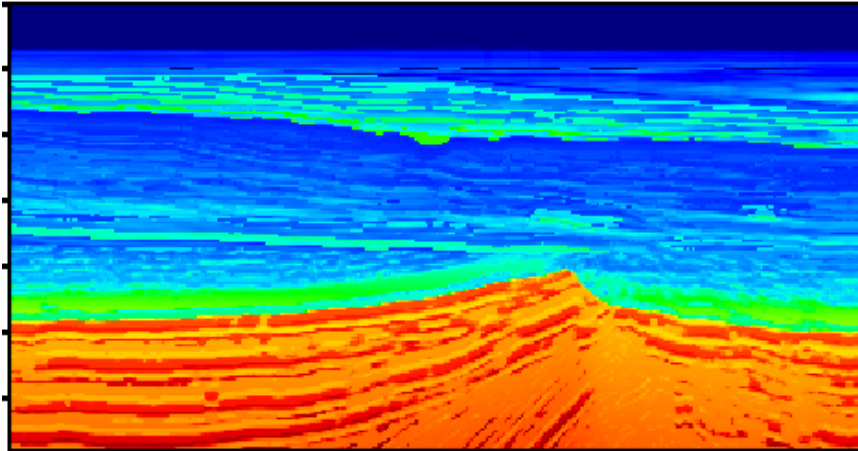


¹Choi et al. (2021)

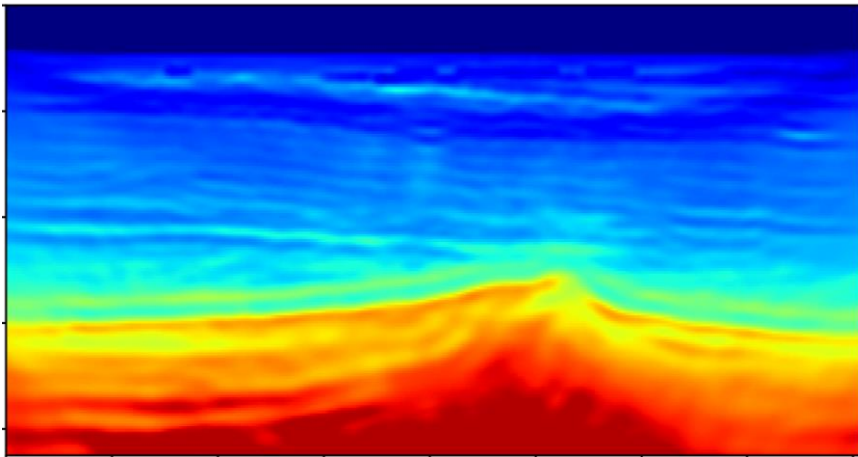


Inverse problems: storing expectations (potential solutions)

Geologically
based
BG
Compass
model



FWI
result



- Faults
- Fractures
- Spatial distribution (increase with)
- Layering
- Reservoir
-

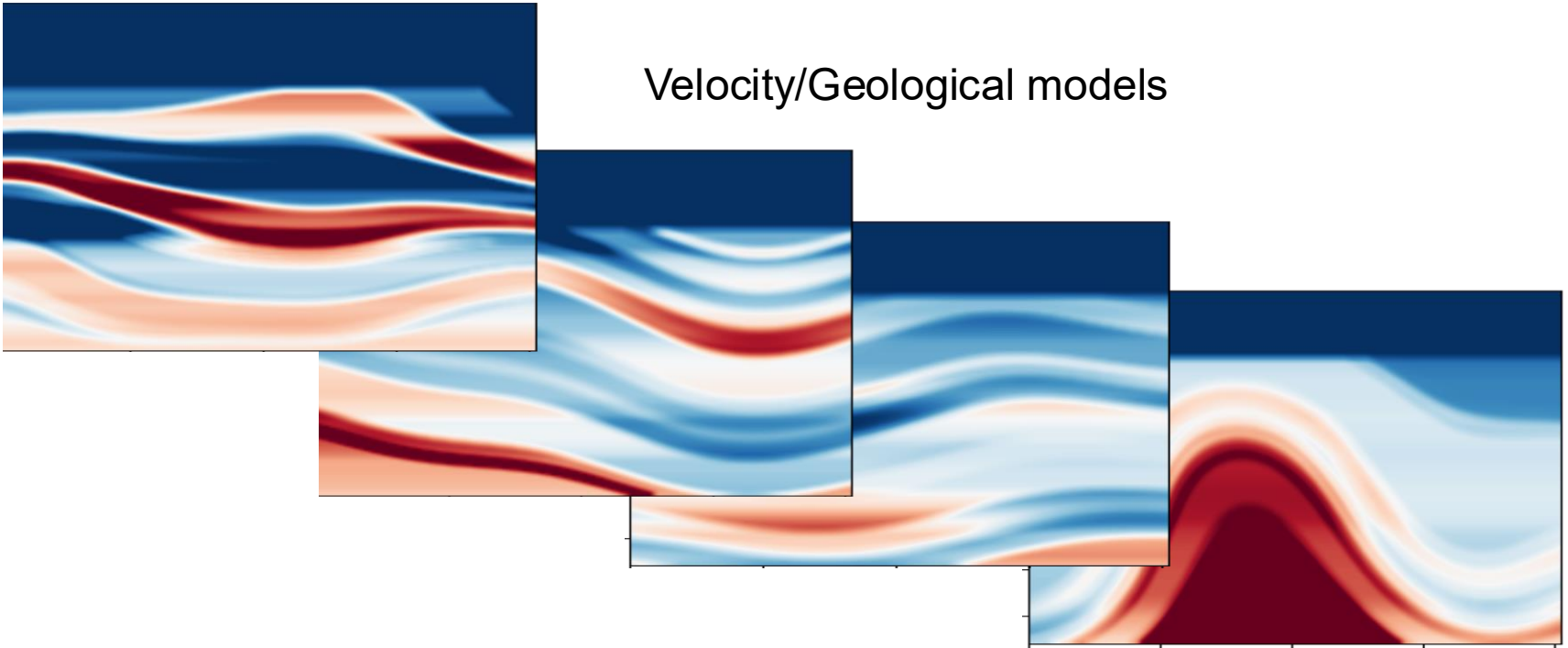


The generative opportunity:

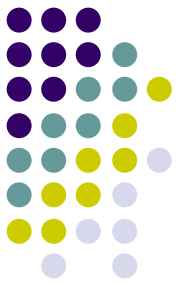
storing a (prior) distribution that covers the potential solution from the solution space in which the network learns the structure of our solution



Velocity/Geological models

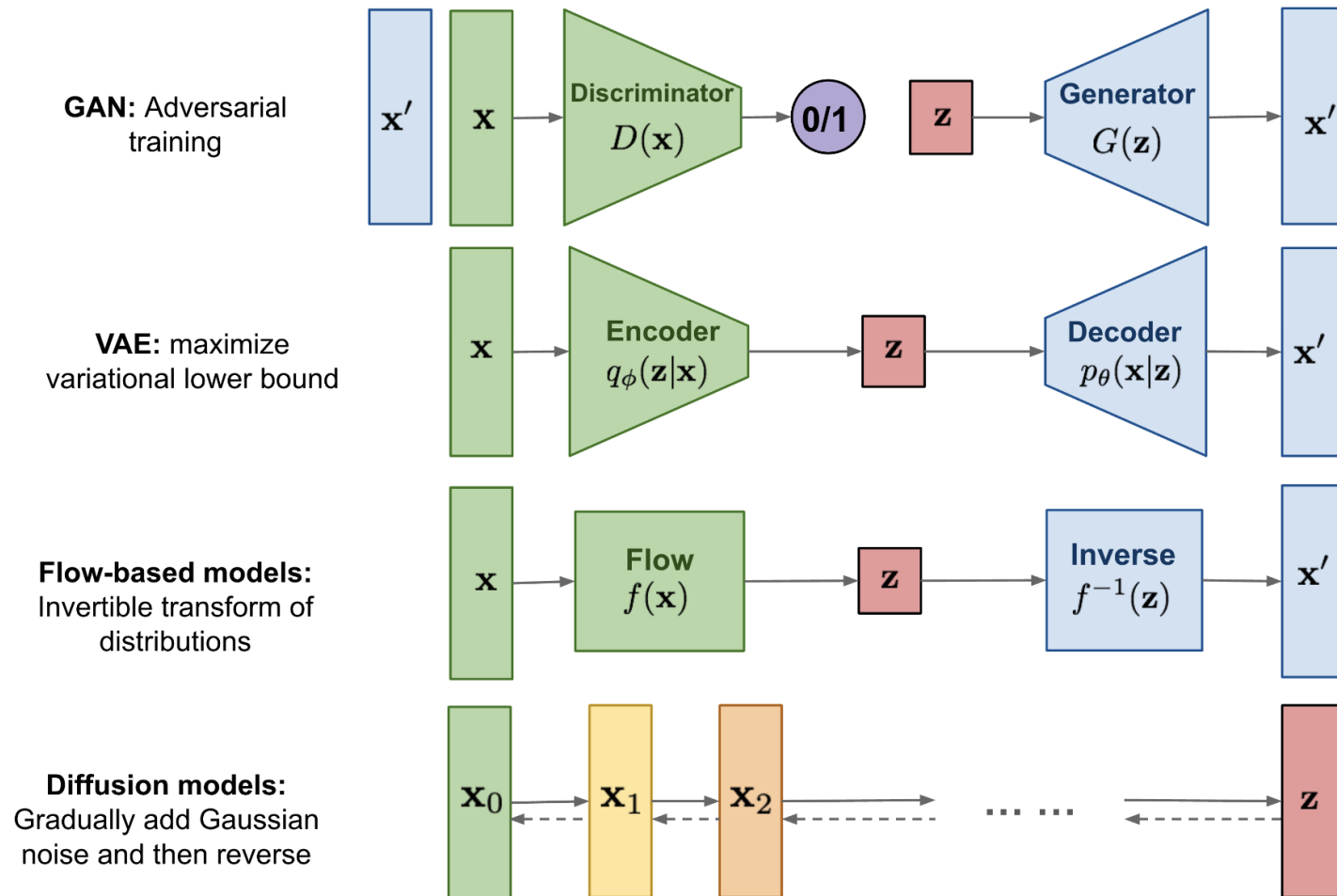
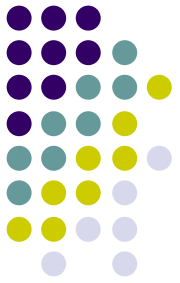


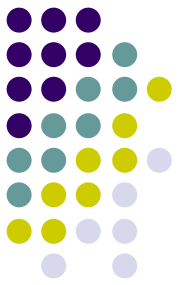
Distributions we store?



- Velocity/Elastic CO2 saturation models (our big objective)
- Salt bodies for flooding and unflooding
- Seismic data (for processing)
- Seismic wavefields
- Location of microseismic events (regression or segmentation)

The generative options:

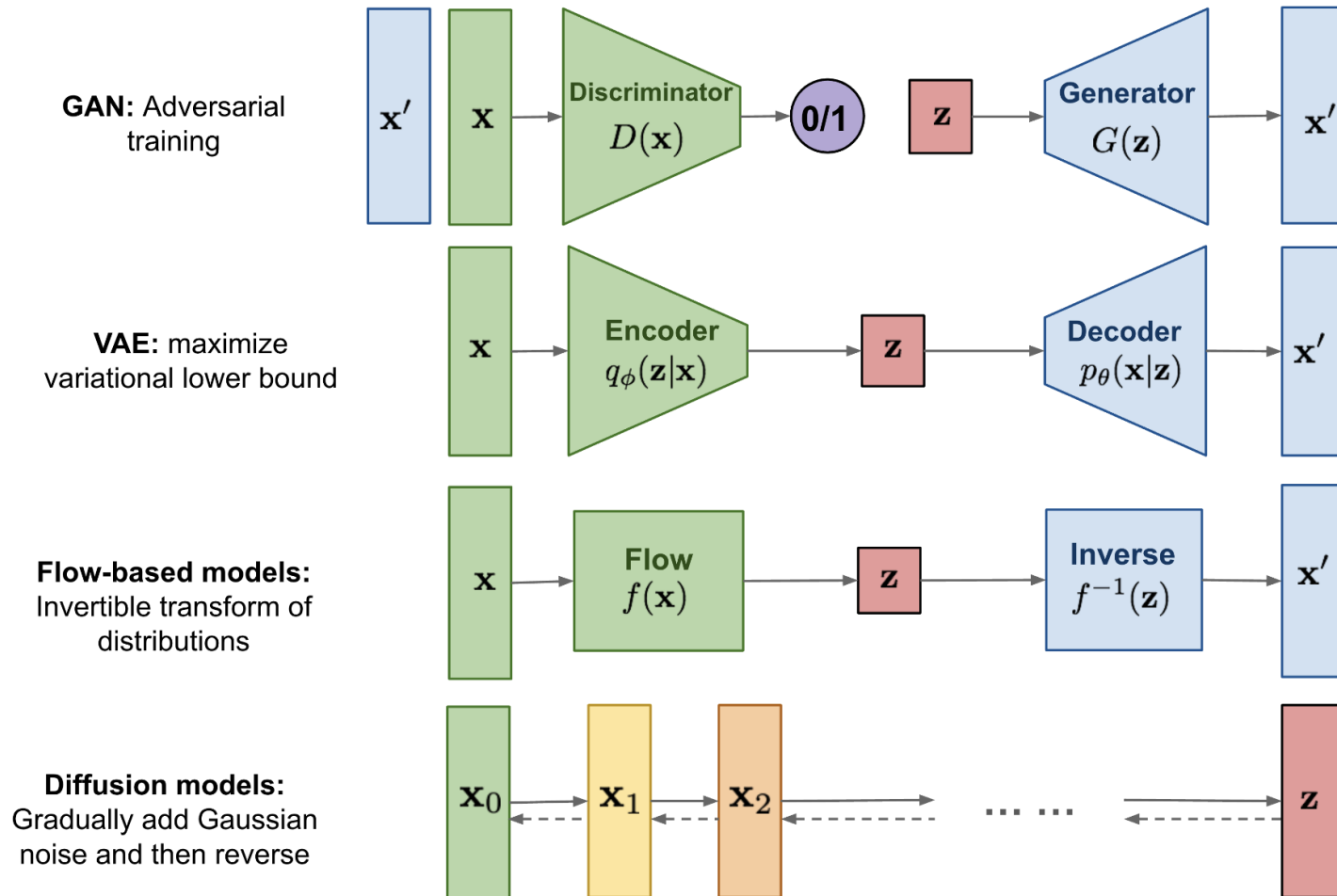
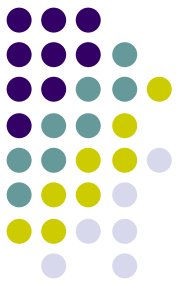


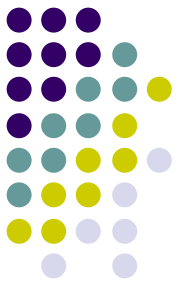


What are Diffusion Models?

- A class of generative models using stochastic processes
- Learn data distribution by reversing a noise process
- Applications: Image, Audio, and Video Synthesis

Sampling from noise:





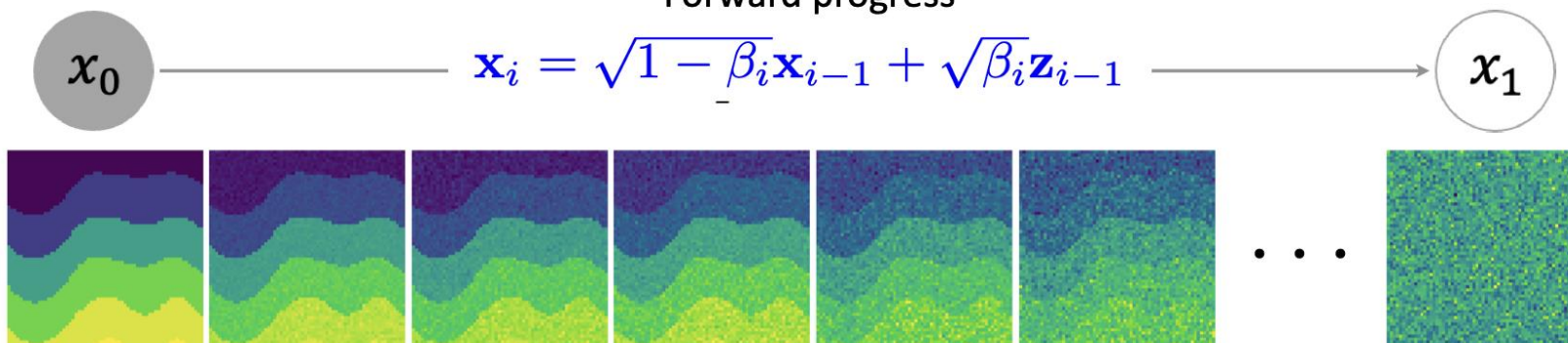
The Diffusion framework

- Another kind of generative models, inspired by physics, based on SDE.
- Convert a well known distribution to the desired learned one through small steps, using the Markov chain
 - Forward Process
 - Reverse Process

The Forward Process (Diffusion)



Forward progress



- **Gradually add Gaussian noise** to data over T steps:

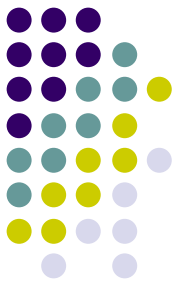
$$q(x_t|x_{t-1}) = \mathcal{N}(x_t; \sqrt{1 - \beta_t}x_{t-1}, \beta_t\mathbf{I})$$

- x_t : Noisy sample at step t .
- β_t : Noise schedule (small positive constants).

- **Closed-form after T steps:**

$$q(x_T|x_0) \approx \mathcal{N}(x_T; 0, \mathbf{I})$$

The Full Forward Process



From data $\mathbf{x}_0$ to noisy image $\mathbf{x}_T$

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Sampling at any step:

$$q(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$$

where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$

In the limit, $\mathbf{x}_\infty$ is a random vector from an isotropic Gaussian

$$\lim_{t \rightarrow \infty} \mathbf{x}_t = \sqrt{\bar{\alpha}_\infty} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_\infty} \boldsymbol{\epsilon} = \boldsymbol{\epsilon} \text{ since } \bar{\alpha}_\infty \rightarrow 0$$



Forward Diffusion: $q(\mathbf{x}_t | \mathbf{x}_0)$

Diffusion forward process (adding Gaussian noise)

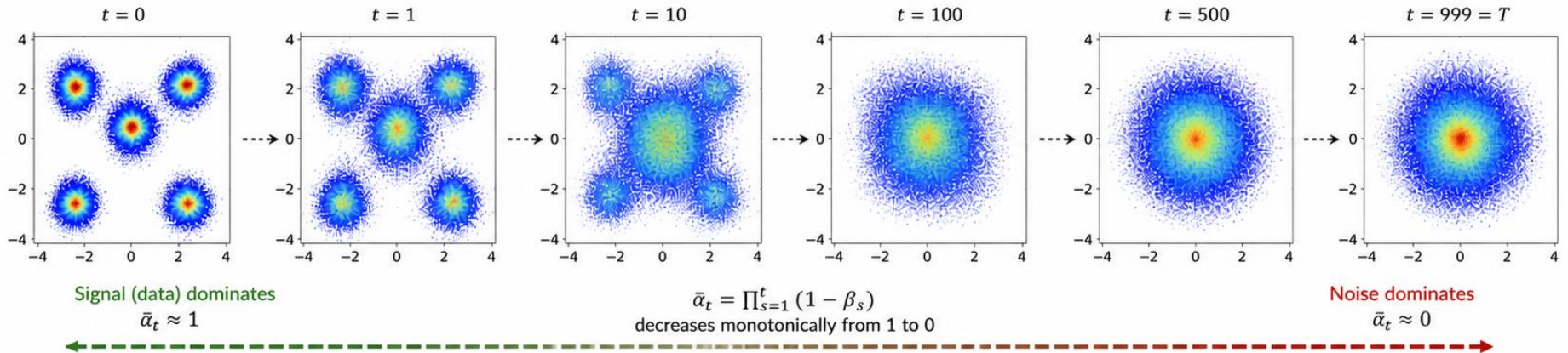
$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1 - \beta_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}) \quad \text{for } t = 1, \dots, T$$

$$\Rightarrow q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}) \quad \text{where } \alpha_t = 1 - \beta_t, \bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

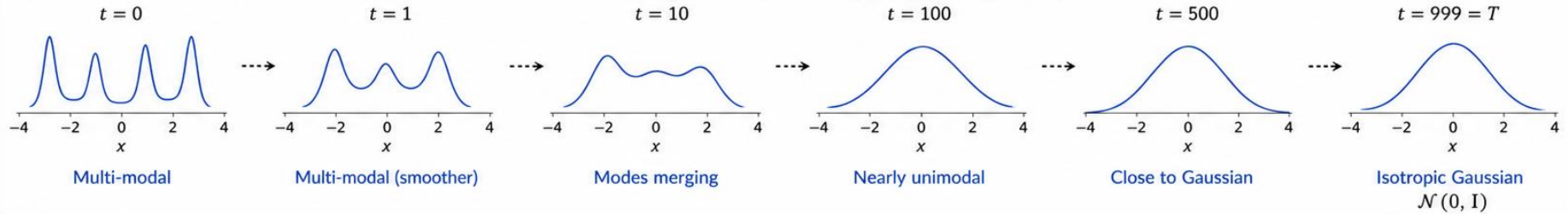
0 (data)

Time t

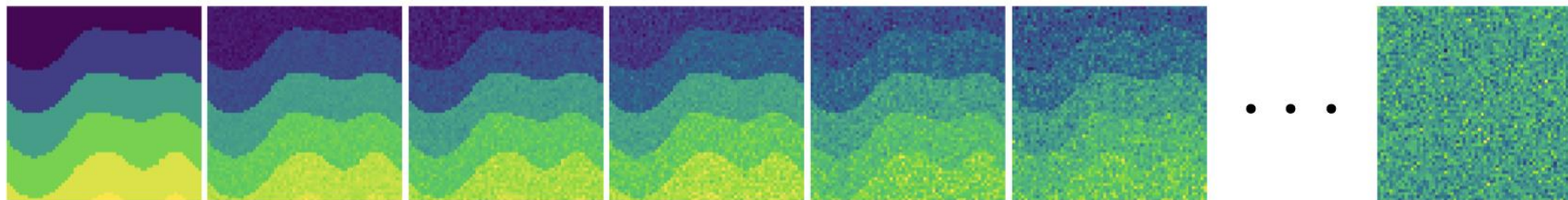
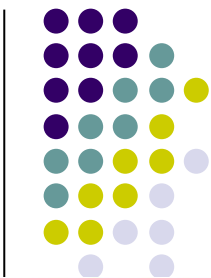
T (noise)



Evolution of Marginal Distribution $q(\mathbf{x}_t)$ (1D projection)



The Reverse Process



$$\begin{array}{c} \textcircled{x_0} \longleftarrow dx = \left[-\frac{\beta(t)}{2}x - \beta(t)\nabla_{x_t} \log p_t(x_t) \right] dt + \sqrt{\beta(t)}d\bar{w} \longrightarrow \textcircled{x_1} \\ \text{Reverse progress} \end{array}$$

- **Goal:** Learn to reverse $q(x_t|x_{t-1})$ by estimating:

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

- θ : Neural network parameters.
- **Key Challenge:** Estimating noise ϵ at each step.



Reverse Diffusion: $p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$

Learn to reverse the noising process and recover data from Gaussian noise

Learned reverse process (parameterized by θ):

$$p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_{\theta}(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}) \quad \text{for } t = T, T-1, \dots, 1$$

Typical mean parameterization:

$$\mu_{\theta}(\mathbf{x}_t, t) = \frac{1}{\sqrt{\bar{\alpha}_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right)$$

β_t : noise schedule

$$\alpha_t = 1 - \beta_t$$

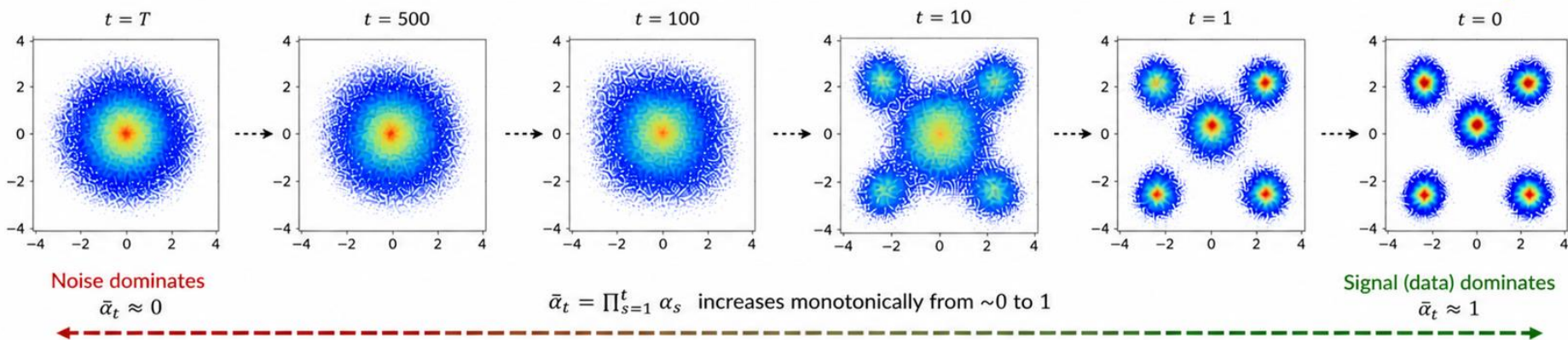
$$\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$$

$\epsilon_{\theta}(\mathbf{x}_t, t)$: noise prediction (NN)

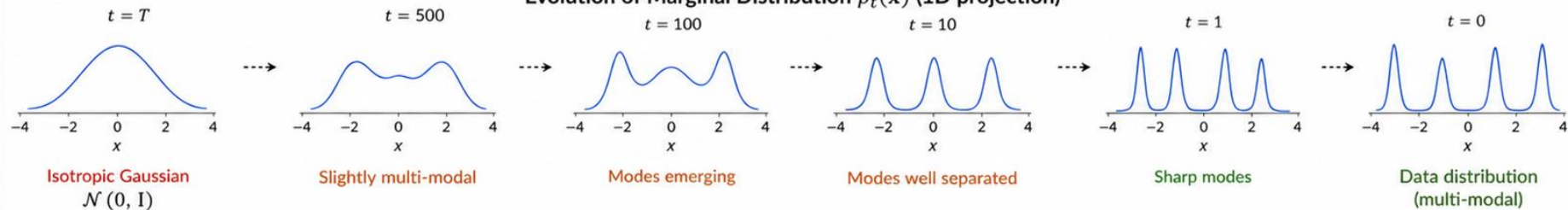
T (noise)

Reverse time t

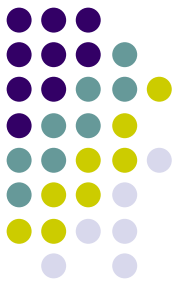
0 (data)



Evolution of Marginal Distribution $p_t(\mathbf{x})$ (1D projection)

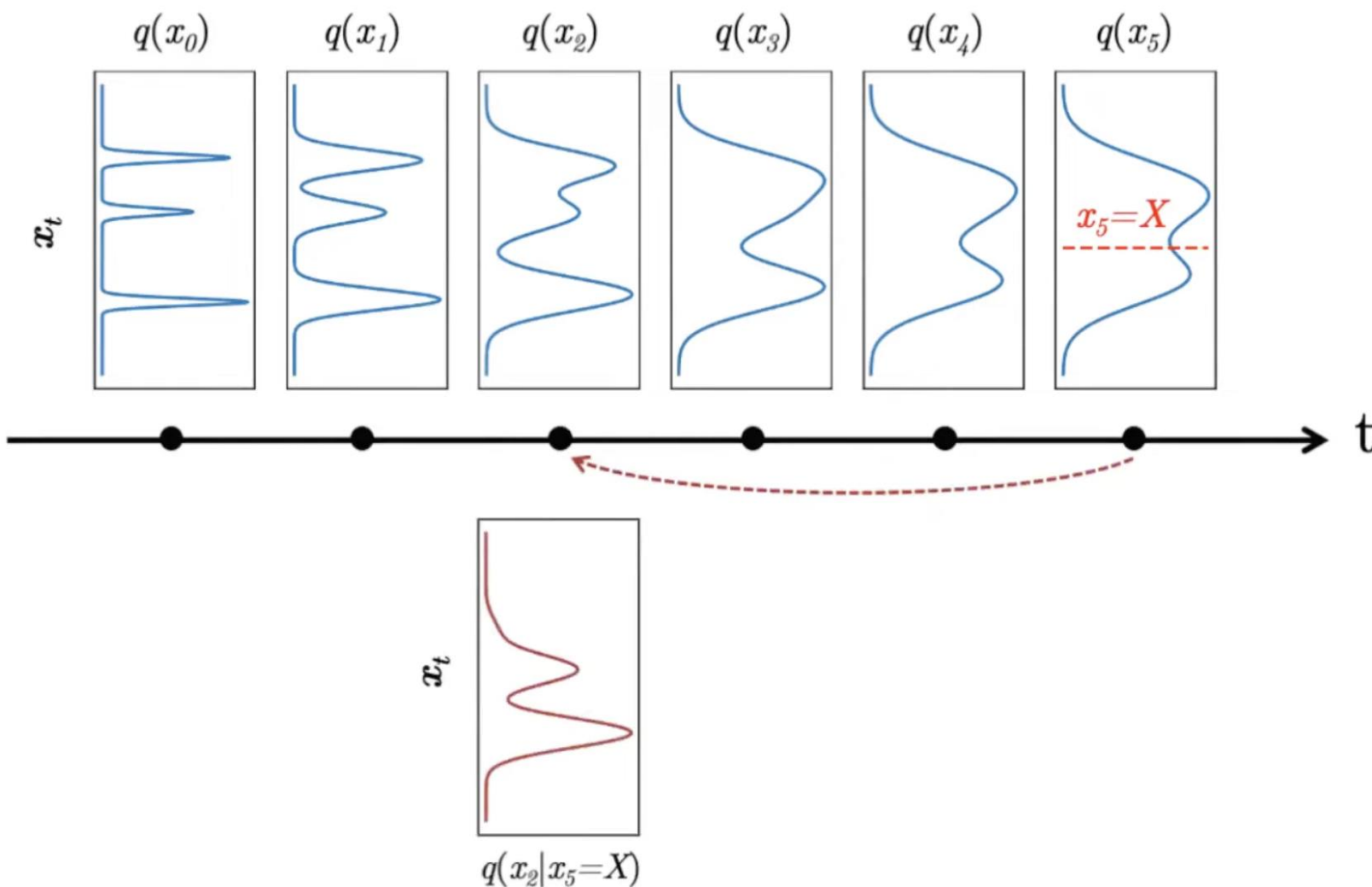


The Chain

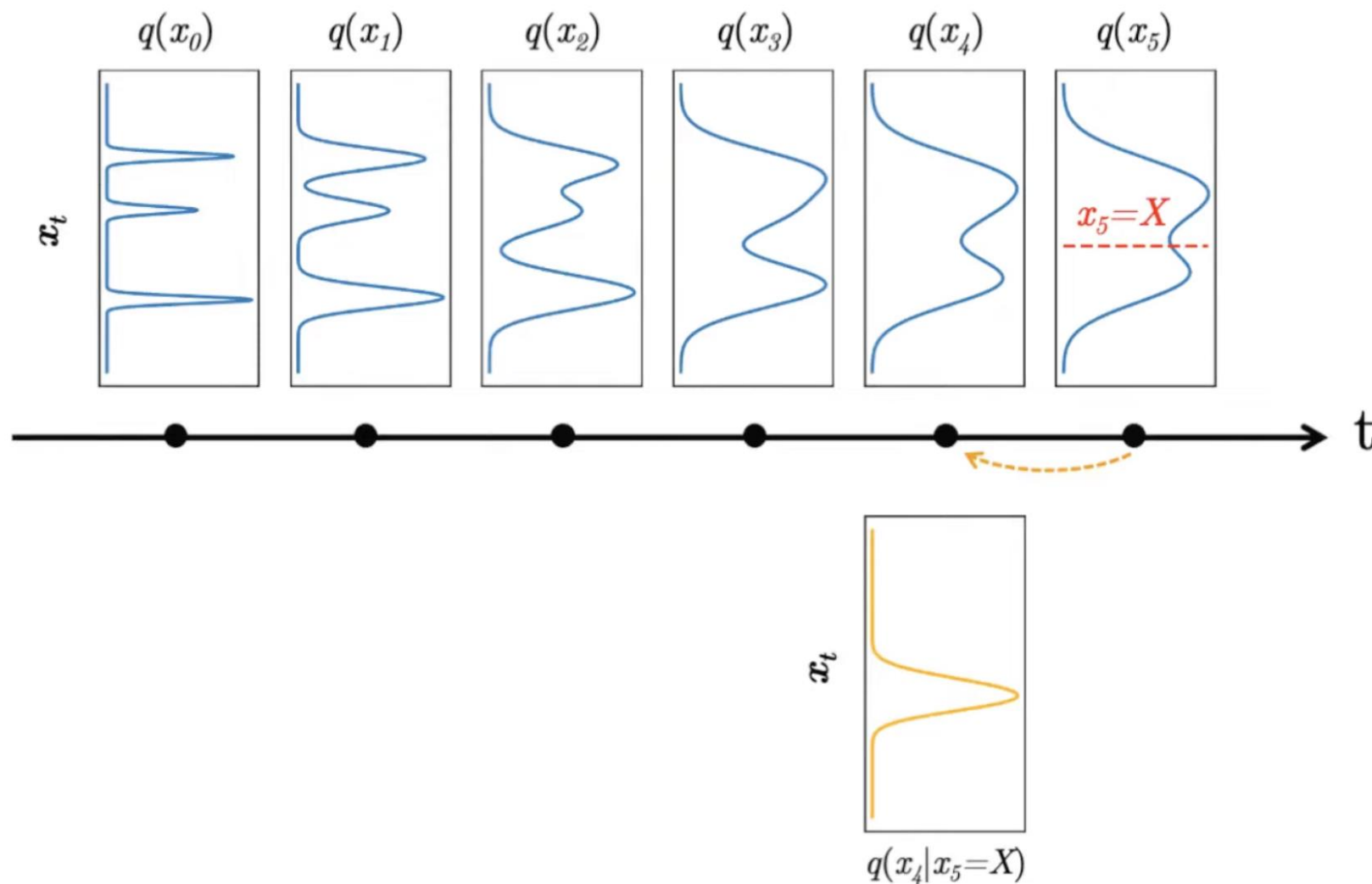


- Forward factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = q(\mathbf{x}_0) \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$
- Reverse factorization: $q(\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_T) = \prod_{t=1}^T q(\mathbf{x}_{t-1} | \mathbf{x}_t) q(\mathbf{x}_T)$
 - Since joint distribution is Gaussian then $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is also Gaussian
 - $q(\mathbf{x}_{t-1} | \mathbf{x}_t) = N(\mathbf{x}_{t-1} | \tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t I)$

One dimension



Normal



The neural approximation



- The reverse conditional $q(\mathbf{x}_{t-1}|\mathbf{x}_t) = N(\mathbf{x}_{t-1}|\tilde{\mu}_t(\mathbf{x}_t, t), \sigma_t \mathbf{I})$ does not have a closed form, but Ho, Jain and Abbeel (2020) derived the following approximation for $\tilde{\mu}_t$:

$$\tilde{\mu}_t(\mathbf{x}_t, t) \approx \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \boldsymbol{\epsilon}_t \right)$$

where $\boldsymbol{\epsilon}_t = N(\boldsymbol{\epsilon}|\mathbf{0}, \mathbf{I})$ is the noise introduced at step t

- We do not know $\boldsymbol{\epsilon}_t$, but we can train a neural network $\epsilon_\theta(\mathbf{x}_t, t)$ to approximate it:

$$\text{Minimize } L(\theta) = \|\boldsymbol{\epsilon}_t - \epsilon_\theta(\mathbf{x}_t, t)\|_2^2$$



Deriving the ELBO Loss for Diffusion Models

Goal: maximize data log-likelihood $\log p_\theta(\mathbf{x}_0) \Leftrightarrow$ minimize negative log-likelihood $-\log p_\theta(\mathbf{x}_0)$

1. Setup

We have a forward (diffusion) process

$$q(\mathbf{x}_{1:T} | \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t | \mathbf{x}_{t-1})$$

and a learned reverse process

$$p_\theta(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$$

where $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$ (Gaussian prior).

2. Start from the negative log-likelihood

$$-\log p_\theta(\mathbf{x}_0)$$

Introduce any distribution $q(\mathbf{x}_{1:T} | \mathbf{x}_0)$ and apply Jensen's inequality:

$$\begin{aligned} -\log p_\theta(\mathbf{x}_0) &= -\log \int p_\theta(\mathbf{x}_{0:T}) d\mathbf{x}_{1:T} \\ &= -\log \int q(\mathbf{x}_{1:T} | \mathbf{x}_0) \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} d\mathbf{x}_{1:T} \\ &\leq -\mathbb{E}_{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] \\ &\triangleq \mathcal{L}_{\text{ELBO}}(\mathbf{x}_0) \end{aligned}$$

3. Expand the ELBO

$$\begin{aligned} \mathcal{L}_{\text{ELBO}}(\mathbf{x}_0) &= -\mathbb{E}_q [\log p_\theta(\mathbf{x}_{0:T}) - \log q(\mathbf{x}_{1:T} | \mathbf{x}_0)] \\ &= -\mathbb{E}_q \left[\log p(\mathbf{x}_T) + \sum_{t=1}^T \log p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) \right. \\ &\quad \left. - \sum_{t=1}^T \log q(\mathbf{x}_t | \mathbf{x}_{t-1}) \right] \\ &= -\mathbb{E}_q [\log p(\mathbf{x}_T)] + \sum_{t=1}^T \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right] \end{aligned}$$

4. Decompose into KL terms

Add and subtract $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ inside the log:

$$\log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} = \log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_{t-1} | \mathbf{x}_t)} + \log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)}$$

Thus,

$$\mathcal{L}_{\text{ELBO}}(\mathbf{x}_0) = \underbrace{-\mathbb{E}_q [\log p(\mathbf{x}_T)]}_{\text{constant (w.r.t. } \theta)} + \underbrace{\sum_{t=1}^T \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right]}_{\mathcal{L}_{\text{const}} \text{ (independent of } \theta)} + \underbrace{\sum_{t=1}^T \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{t-1} | \mathbf{x}_t)}{p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right]}_{\text{KL terms}}$$

$$\mathcal{L}_{\text{ELBO}}(\mathbf{x}_0) = \mathcal{L}_{\text{const}} + \sum_{t=1}^T \mathbb{E}_q [\text{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))]$$

where

- $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ is the true posterior (computable in closed form for diffusion models)
- $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ is the learned reverse transition (e.g., Gaussian with mean from a neural network)

5. The constant term

$$\begin{aligned} \mathcal{L}_{\text{const}} &= -\mathbb{E}_q [\log p(\mathbf{x}_T)] \\ &+ \sum_{t=1}^T \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_t | \mathbf{x}_{t-1})}{q(\mathbf{x}_{t-1} | \mathbf{x}_t)} \right] \end{aligned}$$

is independent of θ ,

so minimizing $\mathcal{L}_{\text{ELBO}}$

is equivalent to minimizing

$$\sum_{t=1}^T \mathbb{E}_q [\text{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))]$$

6. Final ELBO (training objective)

$$\mathcal{L}_{\text{ELBO}}(\mathbf{x}_0) = \text{const}$$

$$\sum_{t=1}^T \mathbb{E}_q [\text{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))]$$

Minimize this w.r.t. θ .

This maximizes a lower bound on $\log p_\theta(\mathbf{x}_0)$.

7. From KL to practical loss (Gaussian case $\rightarrow$ ϵ -prediction)

- In diffusion models, both $q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0)$ and $p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t)$ are Gaussian.
- The KL between Gaussians reduces (up to a constant) to a weighted MSE between means.
- With common parameterization (predicting noise $\epsilon_\theta(\mathbf{x}_t, t)$), the loss becomes:

$$\mathcal{L}_{\text{simple}}(\theta) = \underbrace{\mathbb{E}_{t \sim \mathcal{U}(1, T), \mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(0, \mathbf{I})}}_{\text{sample time } t, \text{ data } \mathbf{x}_0, \text{ noise } \epsilon} \underbrace{\left[w(t) \parallel \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t) \right]_2^2}_{\text{weight (often 1)}} \underbrace{\quad}_{\text{MSE between true and predicted noise}}$$

Summary

- We want to minimize $-\log p_\theta(\mathbf{x}_0)$.
- Using Jensen's inequality gives ELBO.
- ELBO = constant + sum of KL terms.
- Minimizing the sum of KLs trains the reverse model.
- For Gaussian diffusion, this reduces to (weighted) MSE on noise prediction.

Role of the ELBO in Formulating the Training Loss for Diffusion Models

We maximize a variational lower bound (ELBO) on the data log-likelihood $\log p_\theta(\mathbf{x}_0)$.
Minimizing the negative ELBO yields a training loss for the model parameters θ .

1. ELBO on log-likelihood

$$\log p_\theta(\mathbf{x}_0) \geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p_\theta(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \right] \triangleq \mathcal{L}_{\text{ELBO}}$$

$$\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_q[\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)]}_{\text{reconstruction term}} - \underbrace{\sum_{t=2}^T \mathbb{E}_q \left[D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)) \right]}_{(T-1) \text{ KL terms}} - \underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p(\mathbf{x}_T))}_{\text{prior matching}}$$

Reconstruction term

Encourages $\mathbf{x}_1$ to contain information about $\mathbf{x}_0$.

For Gaussian $p_\theta(\mathbf{x}_0|\mathbf{x}_1)$, this becomes an MSE term.

KL terms ($t = 2, \dots, T$)

Force the learned reverse transition $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ to match the true posterior $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$.

Prior matching

Makes the final noised state $\mathbf{x}_T$ close to the prior $p(\mathbf{x}_T) = \mathcal{N}(\mathbf{0}, \mathbf{I})$.
(Does not depend on θ .)

Maximizing the ELBO $\Leftrightarrow$ Minimizing the negative ELBO $-\mathcal{L}_{\text{ELBO}}$

2. From negative ELBO to a practical loss

For the forward process q (fixed) and Gaussian assumptions, the negative ELBO can be simplified (constants independent of θ are dropped):

$$-\mathcal{L}_{\text{ELBO}} = \underbrace{\mathbb{E}_q \|\mathbf{x}_0 - \hat{\mathbf{x}}_0(\mathbf{x}_1)\|_2^2}_{\text{reconstruction (t=1)}} + \sum_{t=2}^T \lambda_t \underbrace{\mathbb{E}_q \|\mu_\theta(\mathbf{x}_t, t) - \mu_q(\mathbf{x}_t, \mathbf{x}_0)\|_2^2}_{\text{KL terms}} + \text{const}$$

- $\mu_q(\mathbf{x}_t, \mathbf{x}_0)$: true posterior mean of $q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)$ (known analytically).
- $\mu_\theta(\mathbf{x}_t, t)$: mean of the learned reverse distribution $p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)$ (model output).
- $\lambda_t > 0$: weighting depending on noise schedule $\{\beta_t\}$.



Using the reparameterization $\mathbf{x}_t = \sqrt{\bar{a}_t} \mathbf{x}_0 + \sqrt{1 - \bar{a}_t} \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, the KL terms reduce to a simple noise prediction loss:

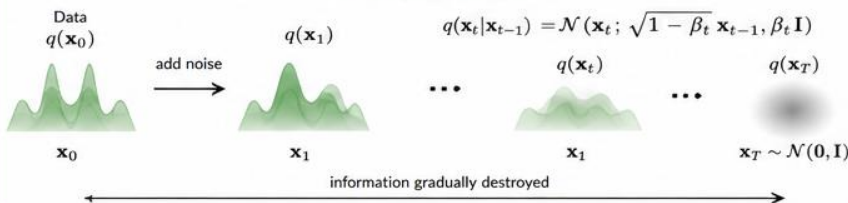
Standard training objective (denoising score matching / ϵ -prediction)

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t \sim \text{Unif}\{1, \dots, T\}, \mathbf{x}_0 \sim q(\mathbf{x}_0), \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})} \left[w_t \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2 \right]$$

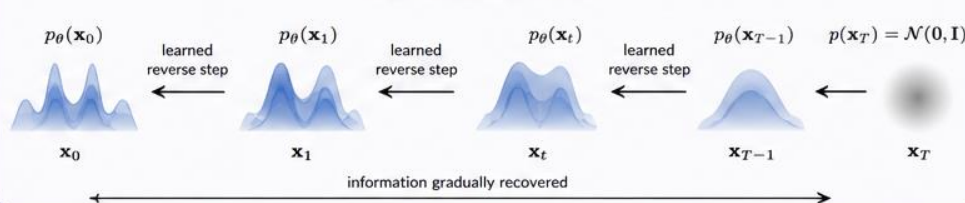
- $\boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)$: model prediction of the noise $\boldsymbol{\epsilon}$ (or equivalently $\mathbf{x}_0$ or the score).
- w_t is a weight derived from λ_t (e.g., $w_t = \frac{\beta_t^2}{2\alpha_t^2(1-\bar{a}_t)\bar{a}_t}$ in the original DDPM).
- Minimizing this loss maximizes a tight ELBO on $\log p_\theta(\mathbf{x}_0)$ up to constants.

3. Intuition: what each term enforces

Forward process (fixed)



Reverse process (learned)



★ The ELBO provides a principled objective whose tractable approximation leads to the simple noise prediction loss widely used to train diffusion models.

The training loss



- **Minimize the variational bound:**

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

- ϵ : Actual noise added.
- ϵ_θ : Predicted noise by the model.
- **Simplified loss:** Predict noise instead of mean μ_θ .

The loss function breaks into simpler components.

Denoising Score Matching:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{t, \mathbf{x}_0, \epsilon} [\|\epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon, t)\|^2]$$

The training Algorithm

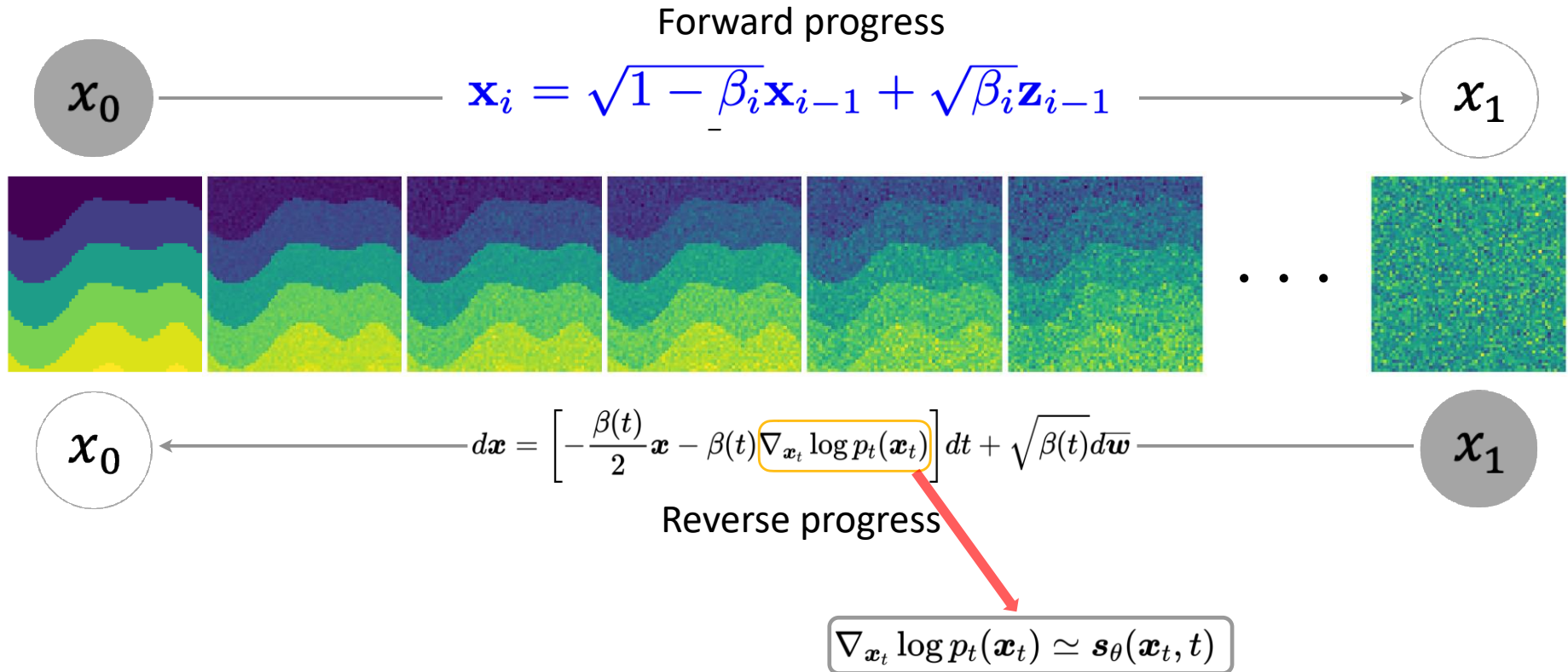
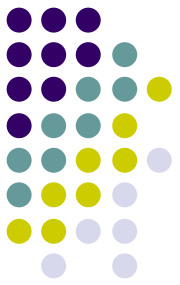


Repeat

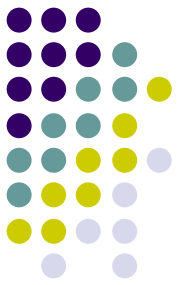
- $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
- $t \sim \text{uniform}(\{1, \dots, T\})$
- $\epsilon \sim N(\mathbf{0}, \mathbf{I})$
- $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$
- $\theta \leftarrow \theta + \eta \nabla_{\theta} ||\epsilon - \epsilon_{\theta}(\mathbf{x}_t, t)||_2^2$

Until convergence

The Diffusion process



The Diffusion training

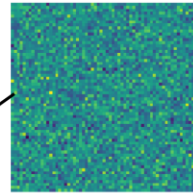


Step 1: Select an original image



x_0

Step 2: Generate Gaussian noise ϵ and time t randomly

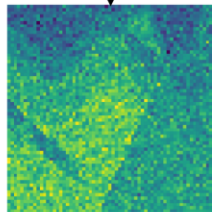


ϵ

$$\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon$$

x_t

Step 3: Corrupt original image

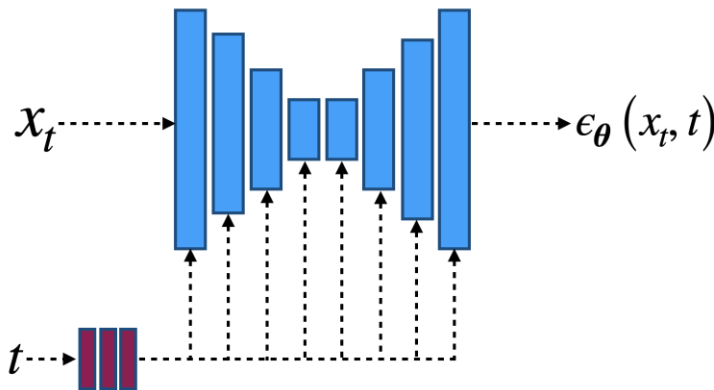


Noise schedule

t

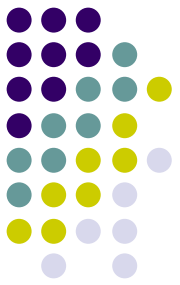
$$\text{Loss} = \left\| \epsilon - \epsilon_{\theta}(x_t, t) \right\|^2$$

Step 4: Calculate loss, use gradient to update U-Net



Fully-connected layers

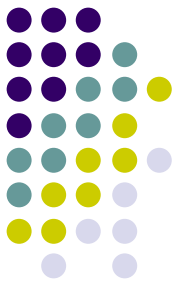
The sampling



- **Start from pure noise** $x_T \sim \mathcal{N}(0, \mathbf{I})$.
- **Iteratively denoise using the learned model:**

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right) + \sigma_t z$$

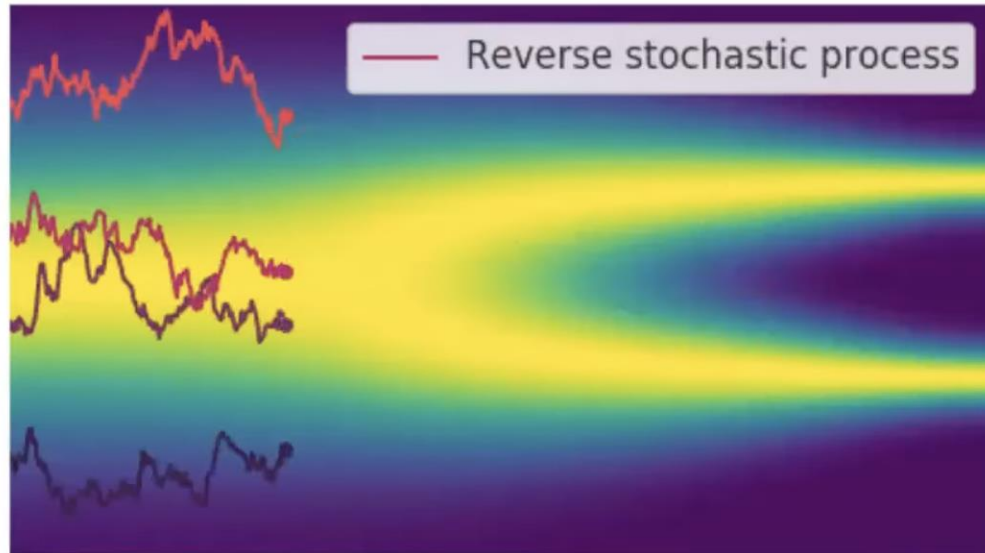
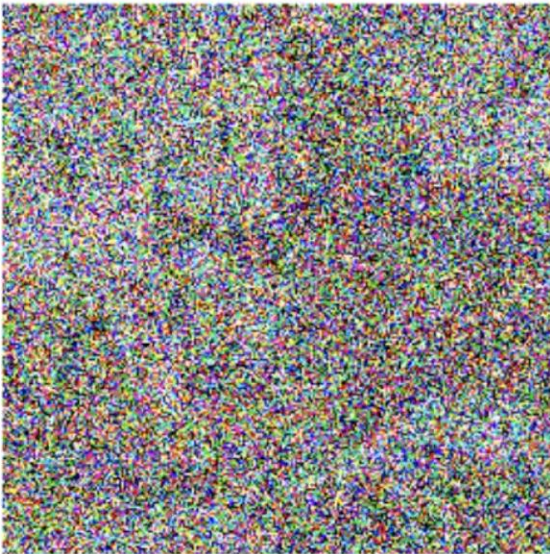
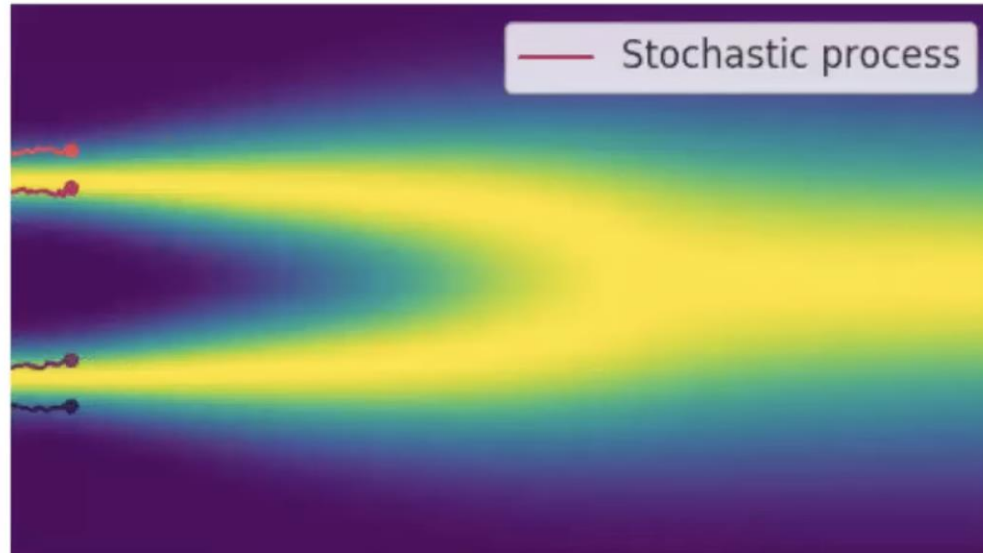
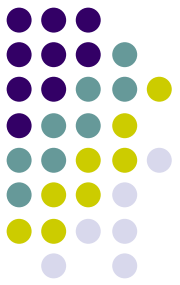
- $\alpha_t = 1 - \beta_t$, $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$.
- $z \sim \mathcal{N}(0, \mathbf{I})$ (optional stochastic term).



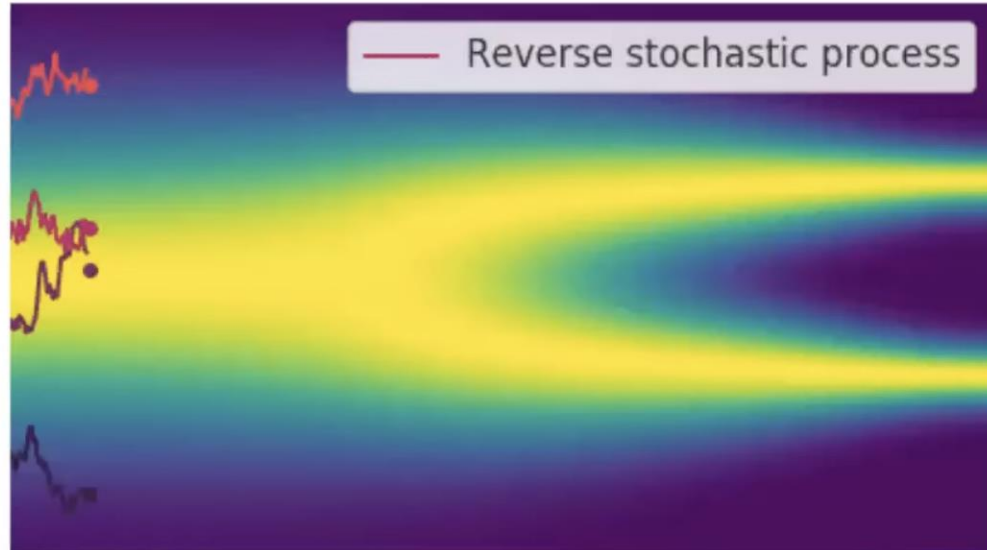
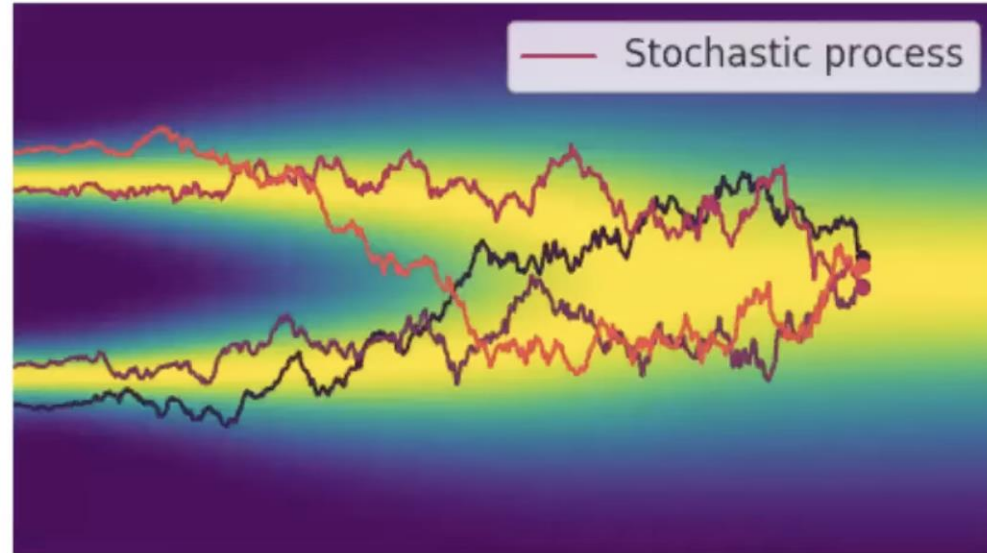
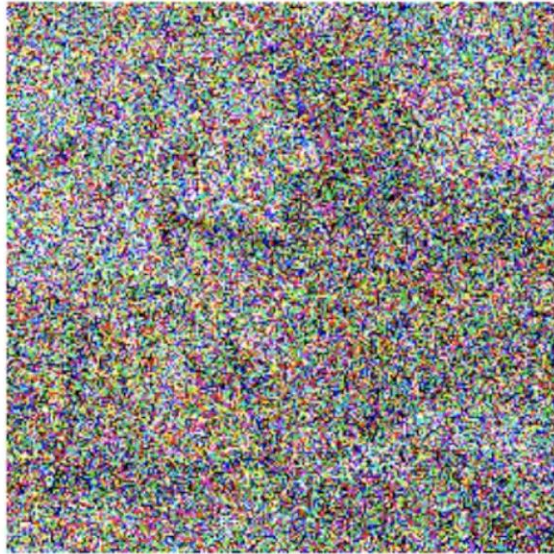
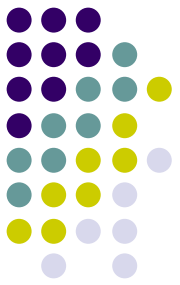
The inference algorithm

- $\mathbf{x}_T \sim N(\mathbf{x}|\mathbf{0}, I)$
- For $t = T, \dots, 1$ do
 - $\epsilon \sim N(\epsilon|\mathbf{0}, I)$ if $t > 1$, else $\epsilon = \mathbf{0}$
 - $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sqrt{\sigma_t} \epsilon$
- Return $\mathbf{x}_0$

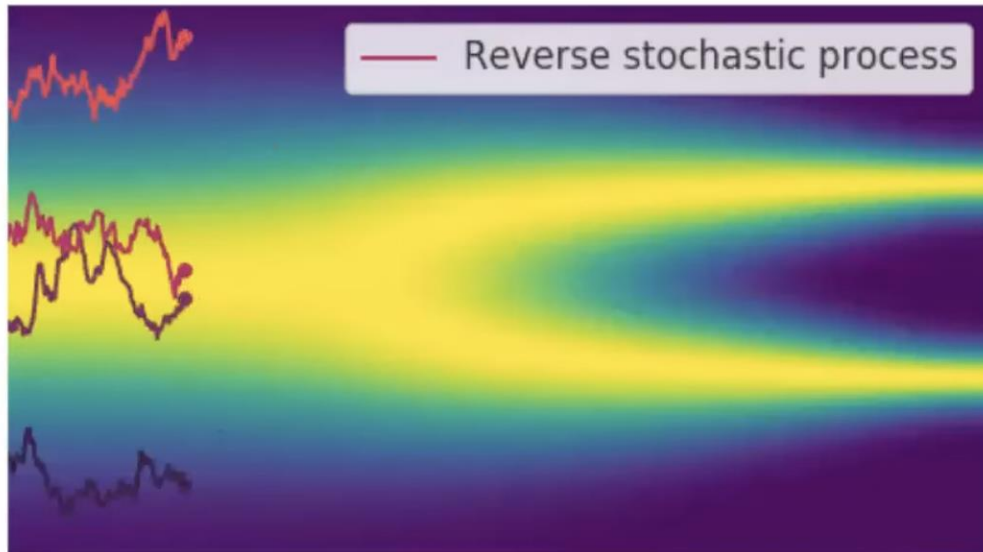
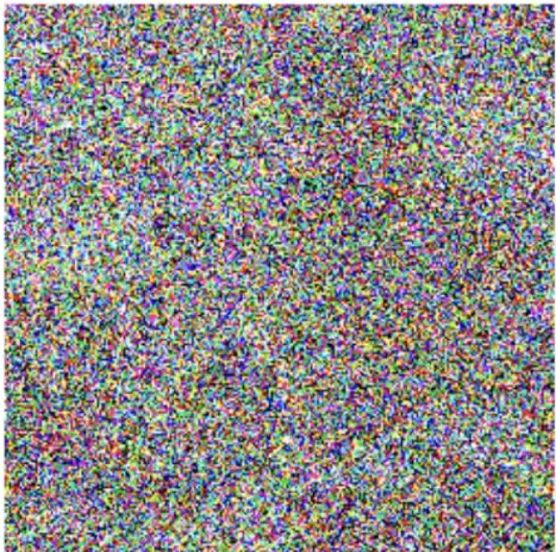
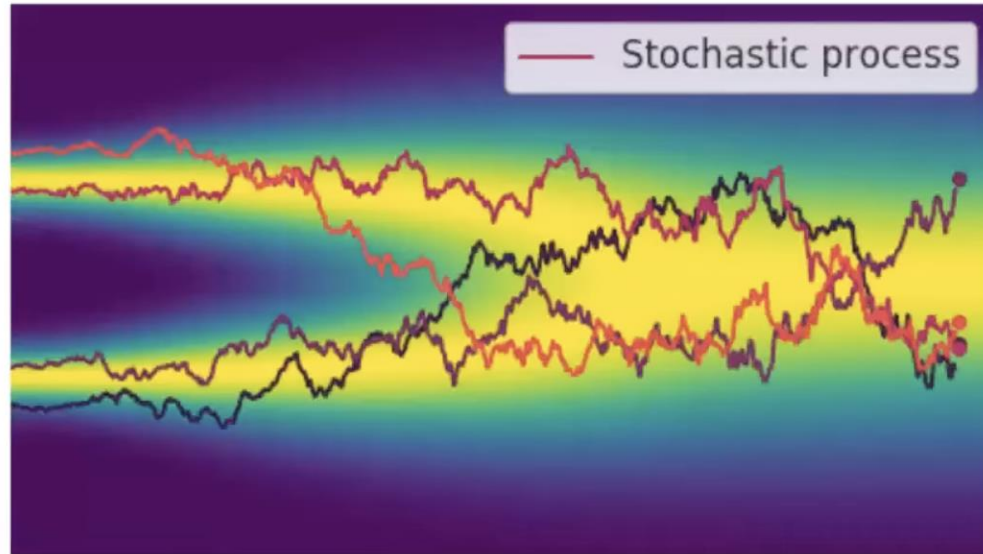
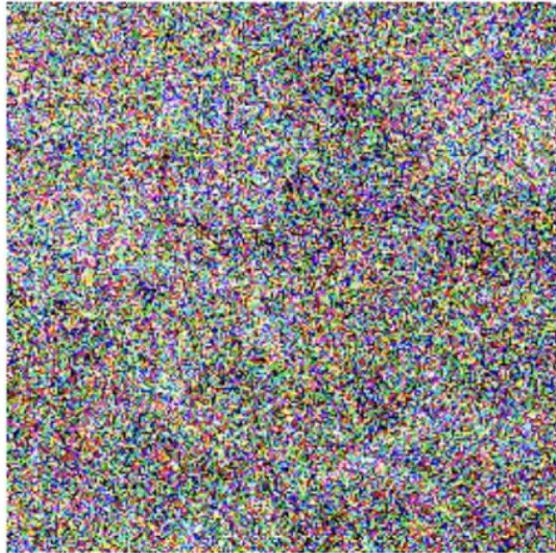
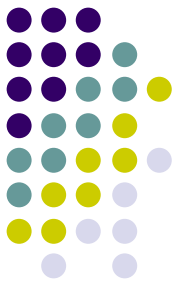
Mapping distributions



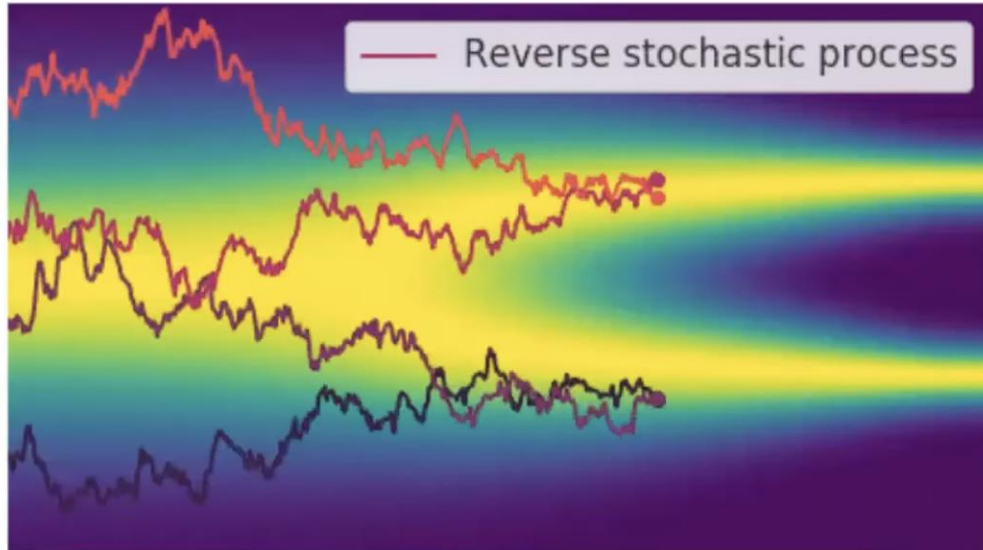
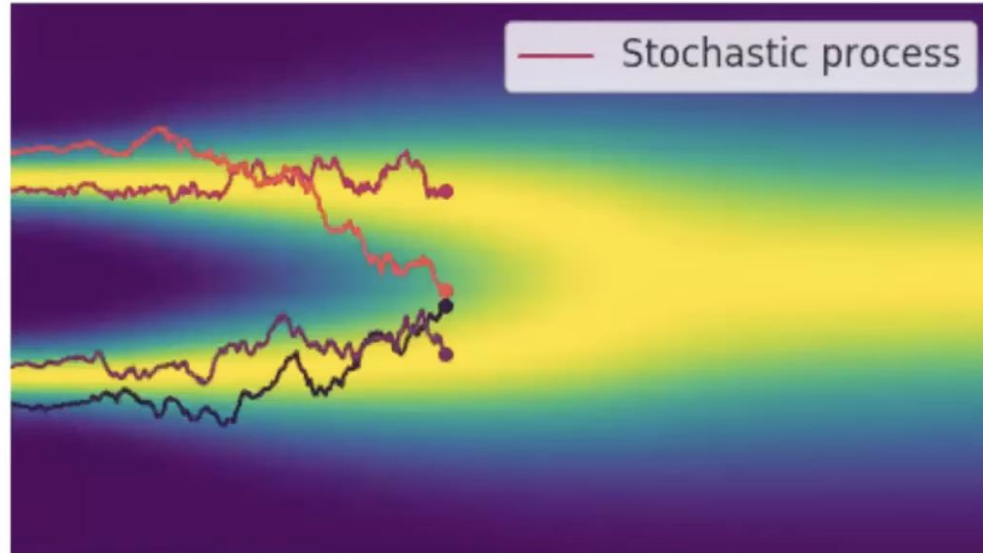
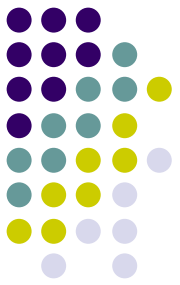
Mapping distributions



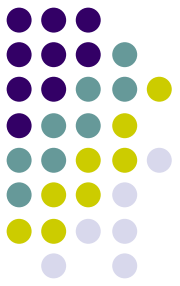
Mapping distributions



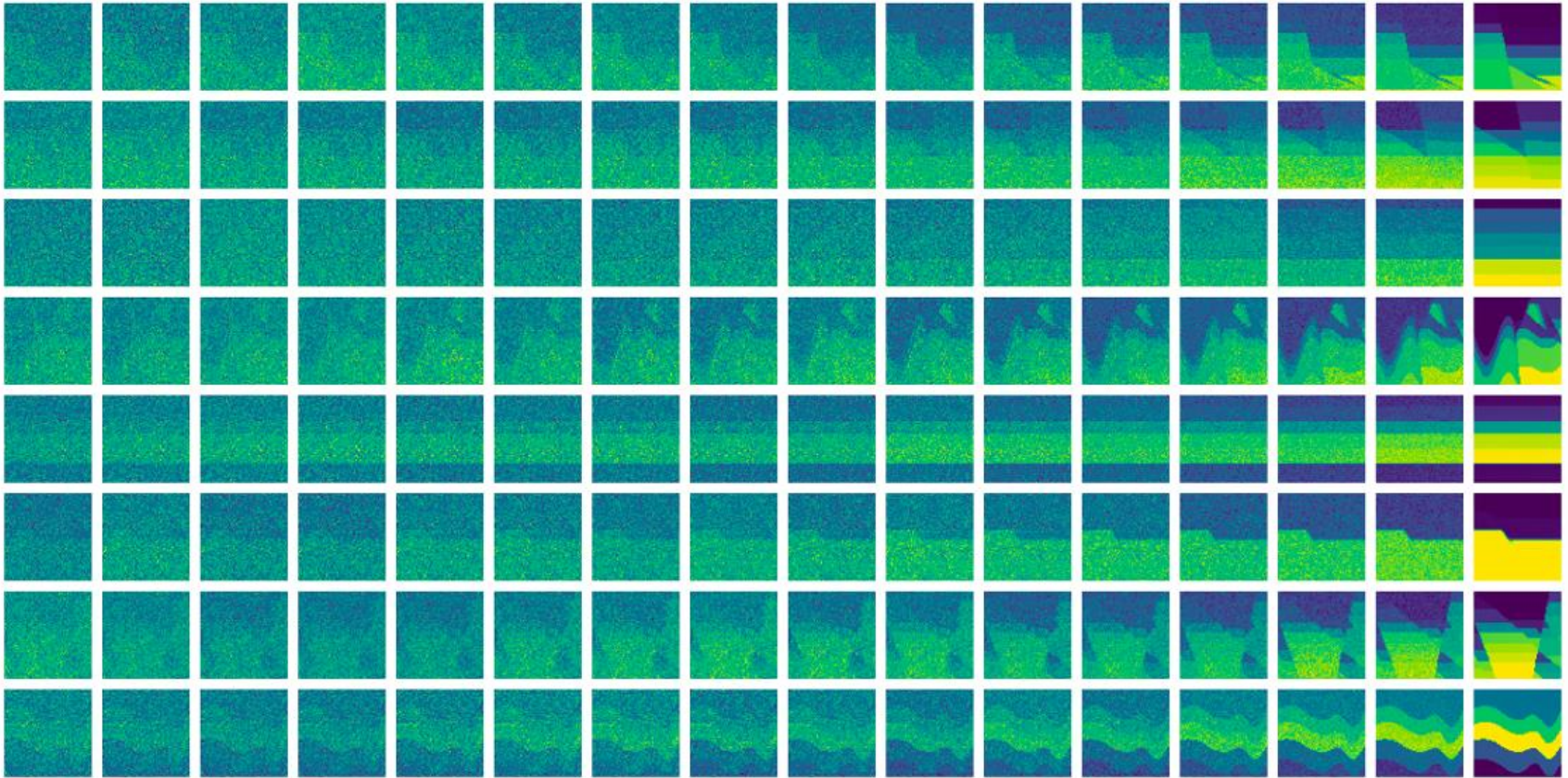
Mapping distributions



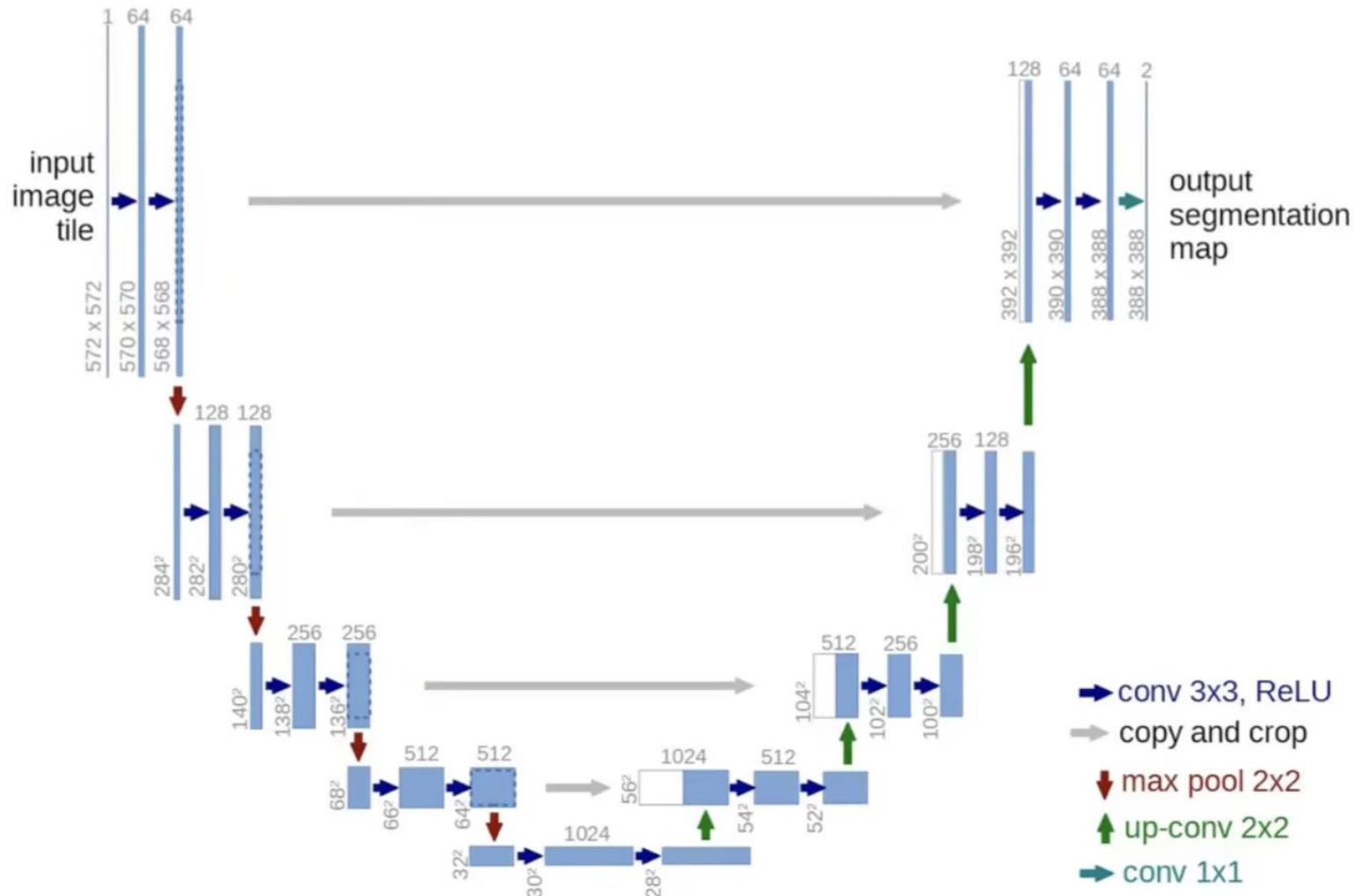
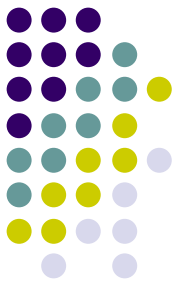
The reverse diffusion progress



Here shows the velocity generation progress every 8 time steps in the final 120 timesteps

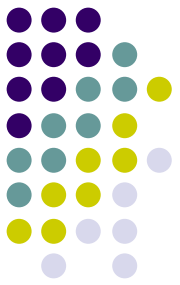


A Unet

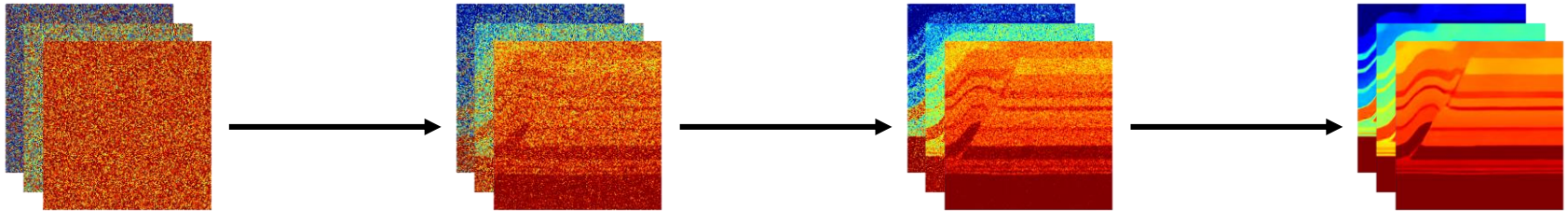


Same size input and output

Elastic velocity model



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Classifier-free guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_i$$



Unconditional

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1 - \beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i, C)) + \sqrt{\beta_i} \mathbf{z}_i$$

Uncertainty? and Generalization?

Reconstruction guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



Unconditional

$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

$$\mathbf{x}_{i-1} = \underbrace{\frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i))}_{\text{Diffusion}} + \underbrace{\sqrt{\beta_i} \mathbf{z}_i}_{\text{Noise}} + \underbrace{\nabla_{\mathbf{x}_i} (\mathcal{F}(\mathbf{x}_i), \mathcal{D})}_{\text{Condition Gradient}}$$

Diffusion

Noise

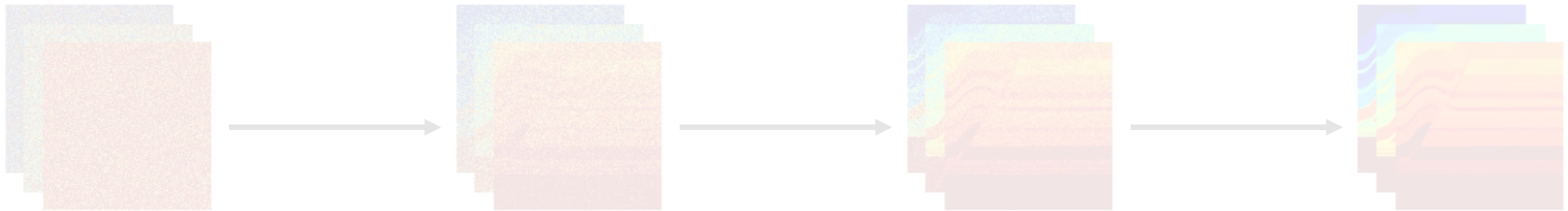
Condition Gradient

Generalization?

Reconstruction guidance



$$\mathbf{x}_i = \sqrt{1 - \beta_i} \mathbf{x}_{i-1} + \sqrt{\beta_i} \mathbf{z}_{i-1}$$



Unconditional

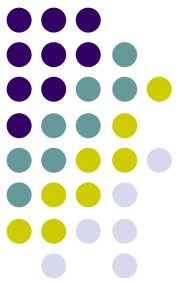
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i$$

Conditional

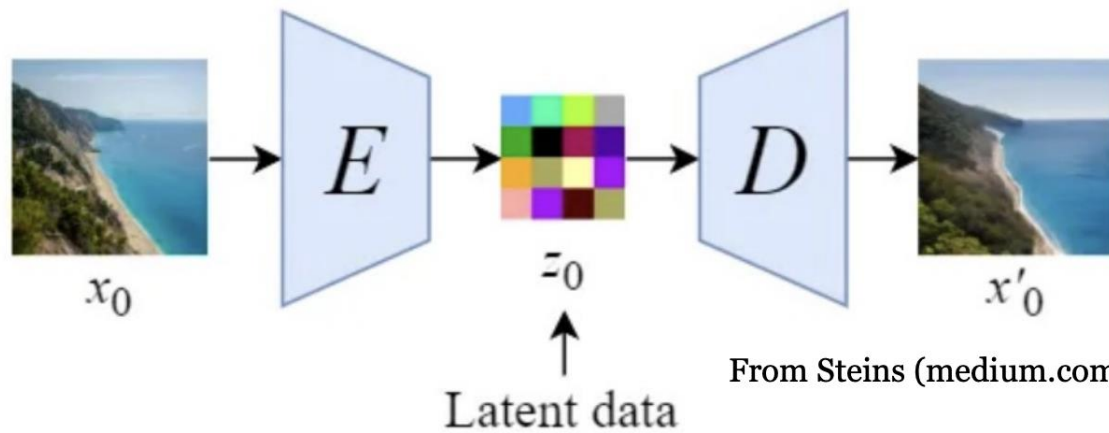
$$\mathbf{x}_{i-1} = \frac{1}{\sqrt{1-\beta_i}} (\mathbf{x}_i + \beta_i \mathbf{s}_{\theta^*}(\mathbf{x}_i, i)) + \sqrt{\beta_i} \mathbf{z}_i + \nabla_{\mathbf{x}_i} (\mathcal{F}(\mathbf{x}_i), \mathcal{D})$$

- ✓ How many gradient steps should be used?
- ✓ How does it compare with the cost of unconditional sampling?

Latent Diffusion Models (Stable Diffusion)

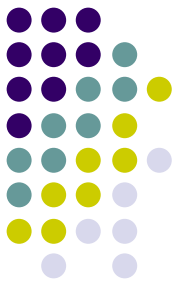


- Rombach, Blattman et al., 2022
- **Speed up:** performing the diffusion in a low dimensional latent space
- **Conditional generation:** condition denoising on text, images, etc.



From Steins (medium.com)

Latent diffusion



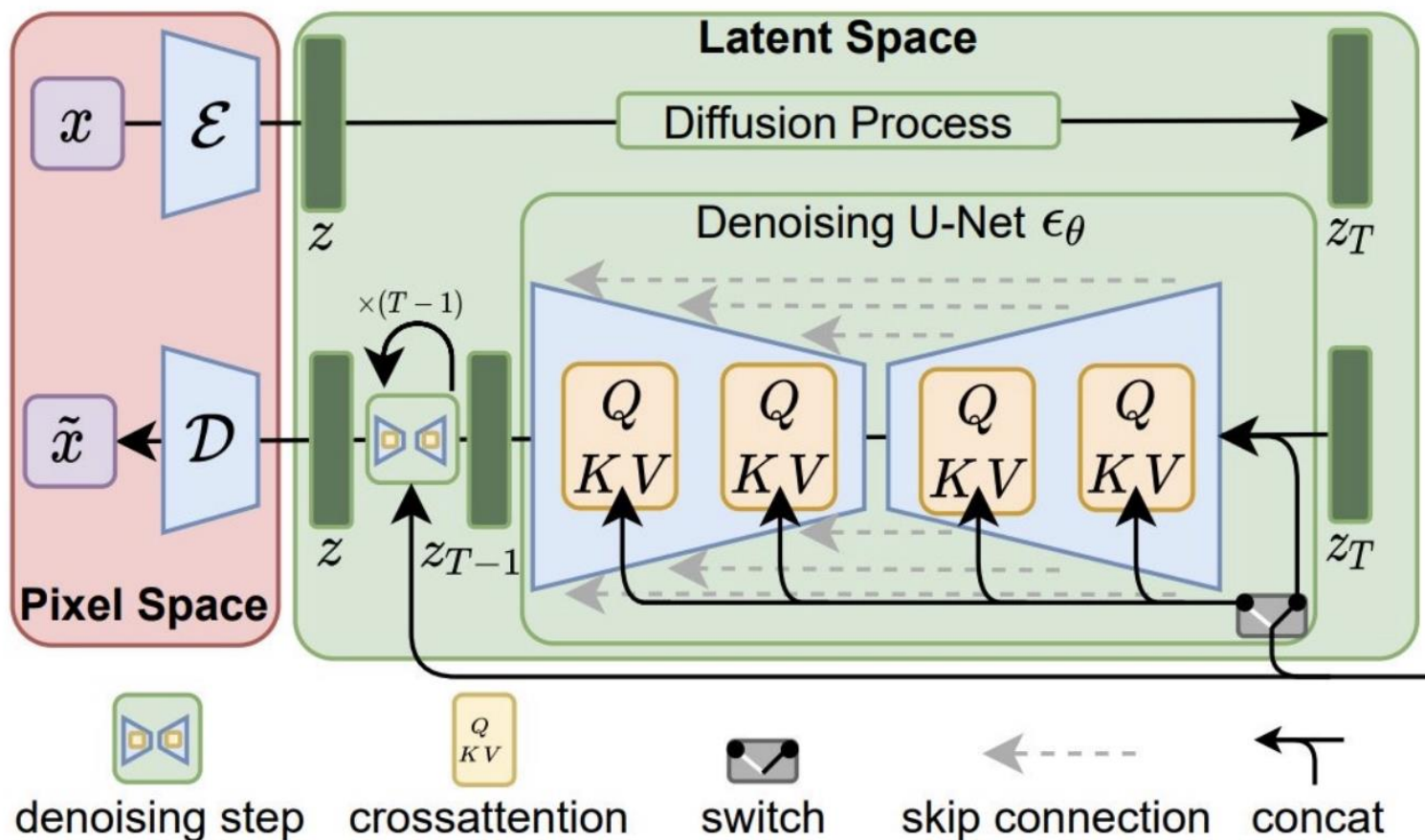
$$\bullet + \Sigma = \bullet$$

Pixel Noise Pixel

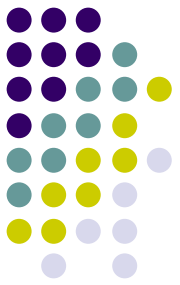
$$\bullet + \Sigma = \bullet$$

Latent Noise Latent

Latent diffusion



Diffusion FWI



The FWI objective function with regularization term:

$$J(\mathbf{m}) = \frac{1}{2} \|\mathbf{d}_{\text{obs}} - \mathbf{F}(\mathbf{m})\|_2^2 + \mathbf{R}(\mathbf{m})$$

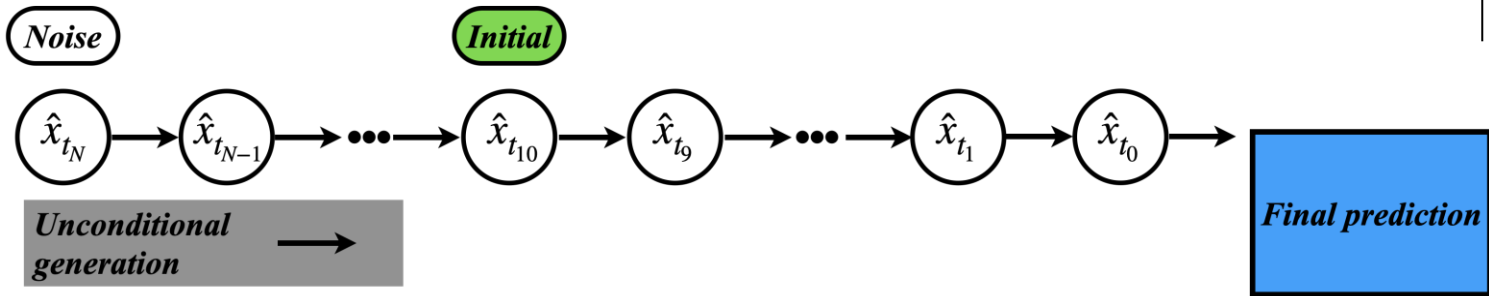
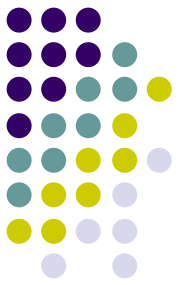
Its solution depends on the proximal operator of the regularization:

$$\mathbf{m}_{k+1} = \text{prox}_{\mathbf{R}} \left(\mathbf{m}_k + \alpha_k \frac{\partial J}{\partial \mathbf{m}_k} \right)$$

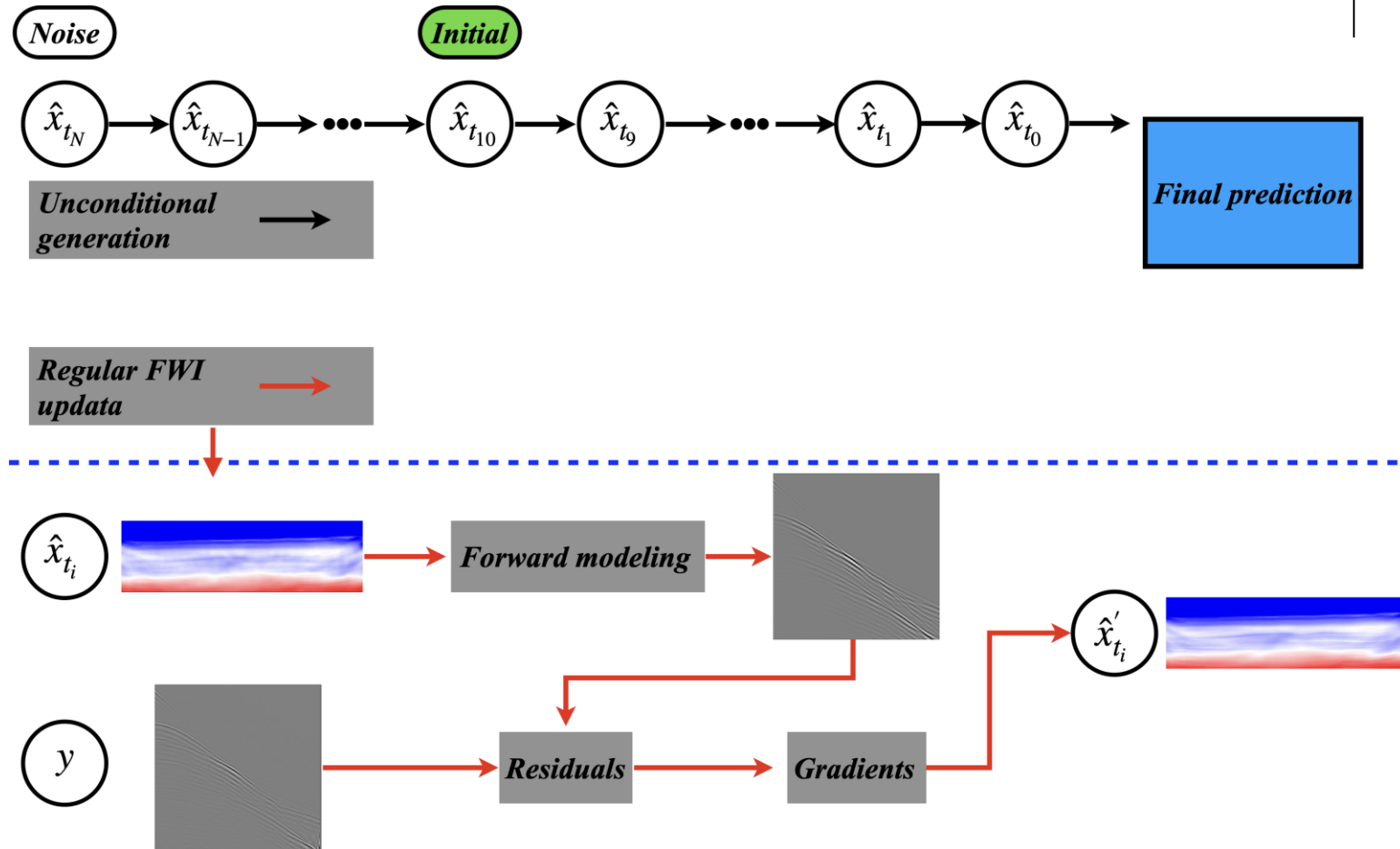
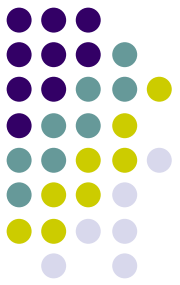
Replace it by reverse Diffusion progress:

$$\mathbf{m}_{k+1} = \mathbf{f} \left(\mathbf{m}_k + \alpha_k \frac{\partial J}{\partial \mathbf{m}_k}, \mathbf{s}_{\theta^*}, t \right) + \sqrt{\beta(t)} d\mathbf{w}$$

Diffusion FWI framework

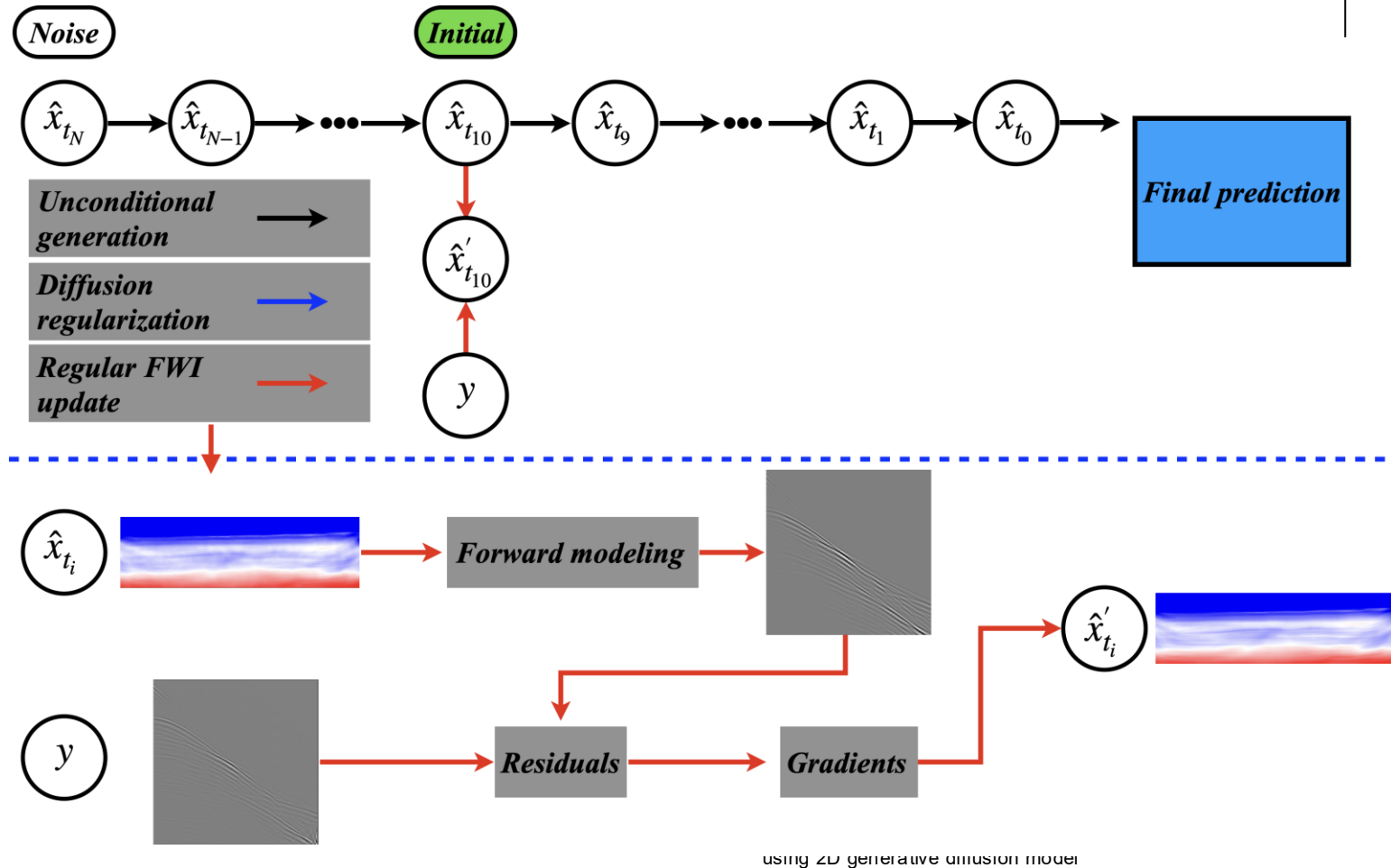
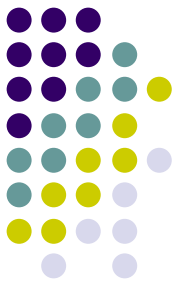


Diffusion FWI framework



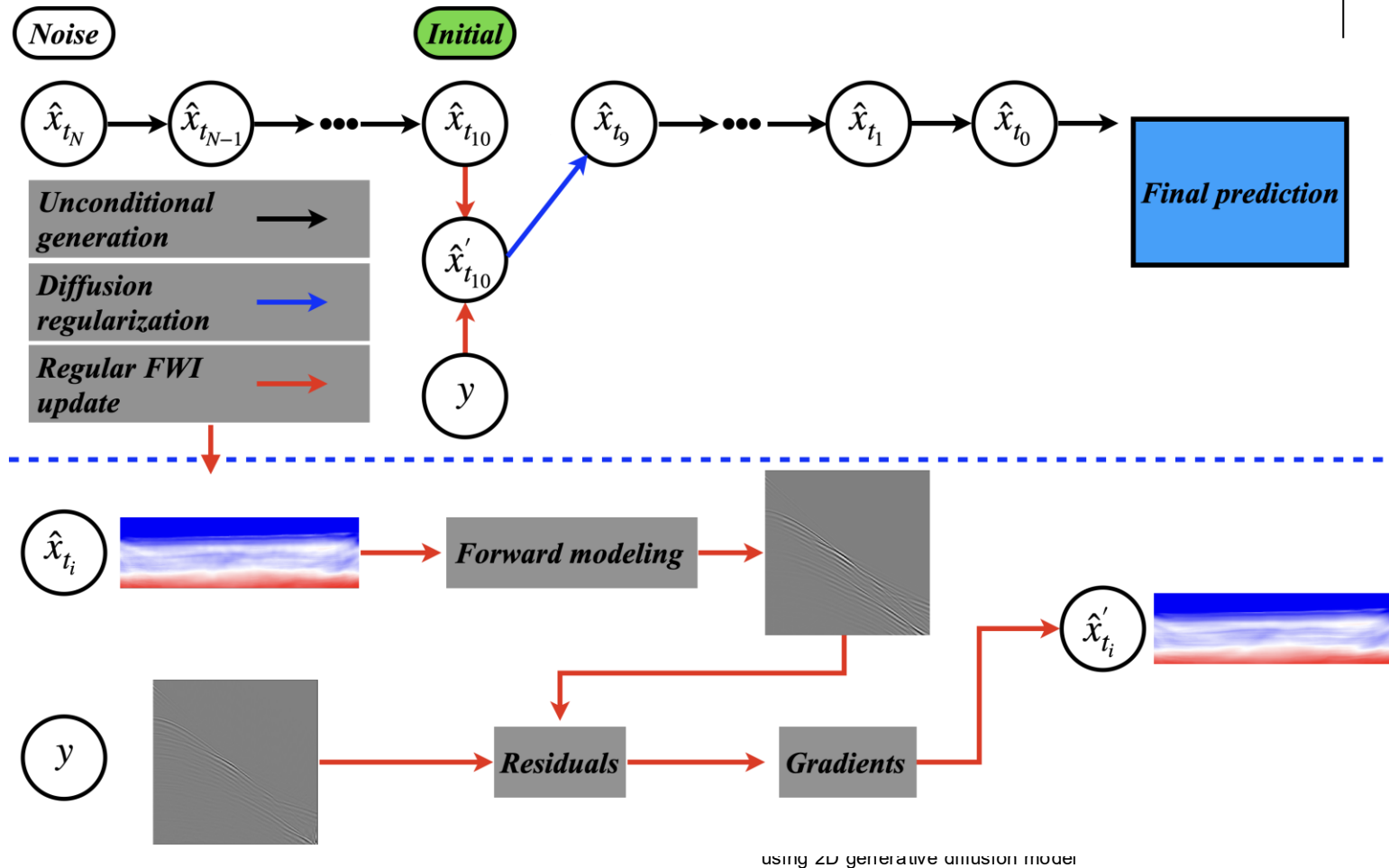
using 2D generative diffusion model

Diffusion FWI framework



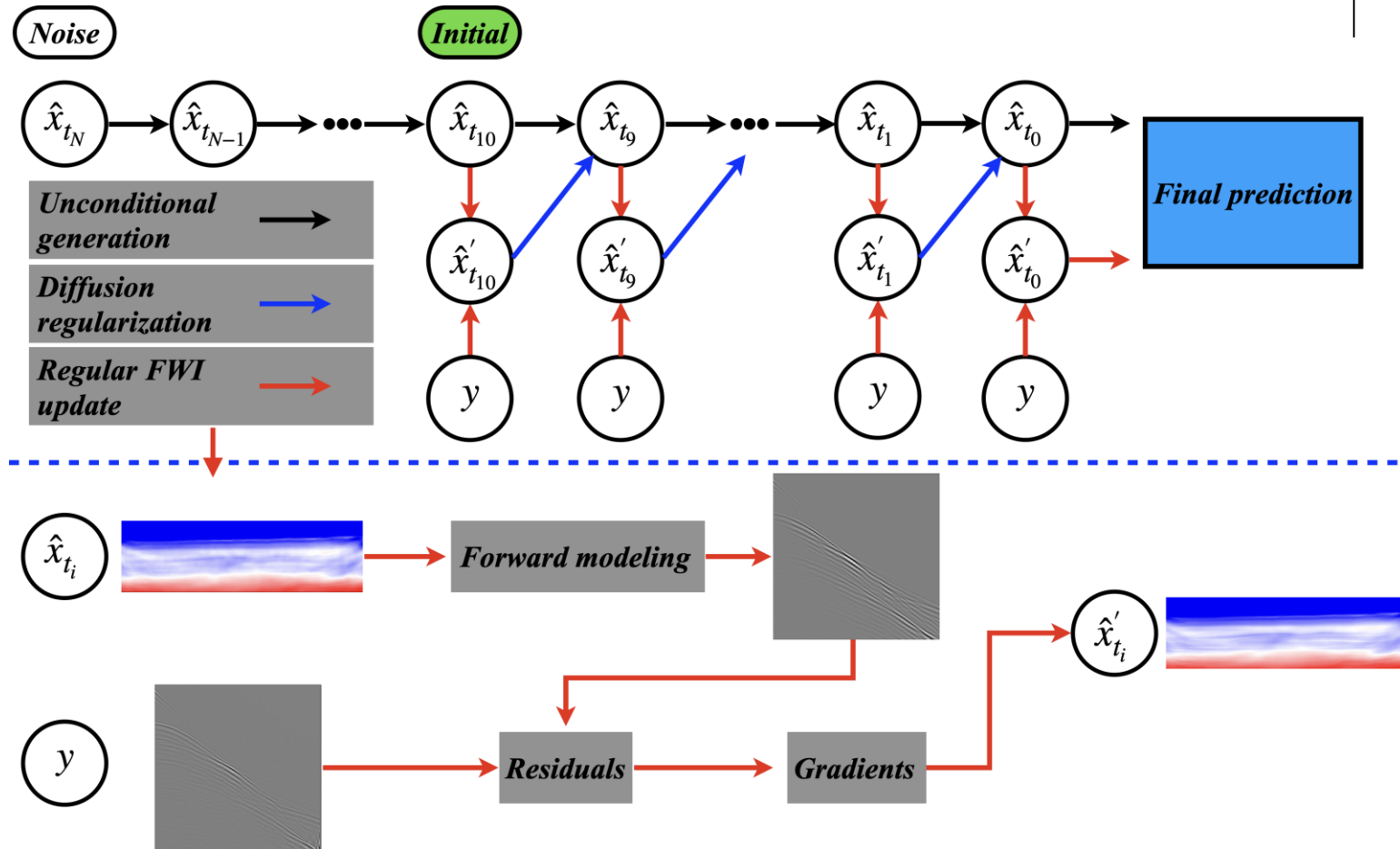
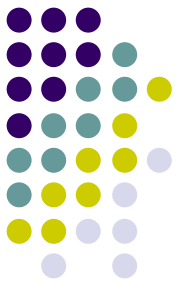
using 2D generative diffusion model

Diffusion FWI framework



using 2D generative diffusion model

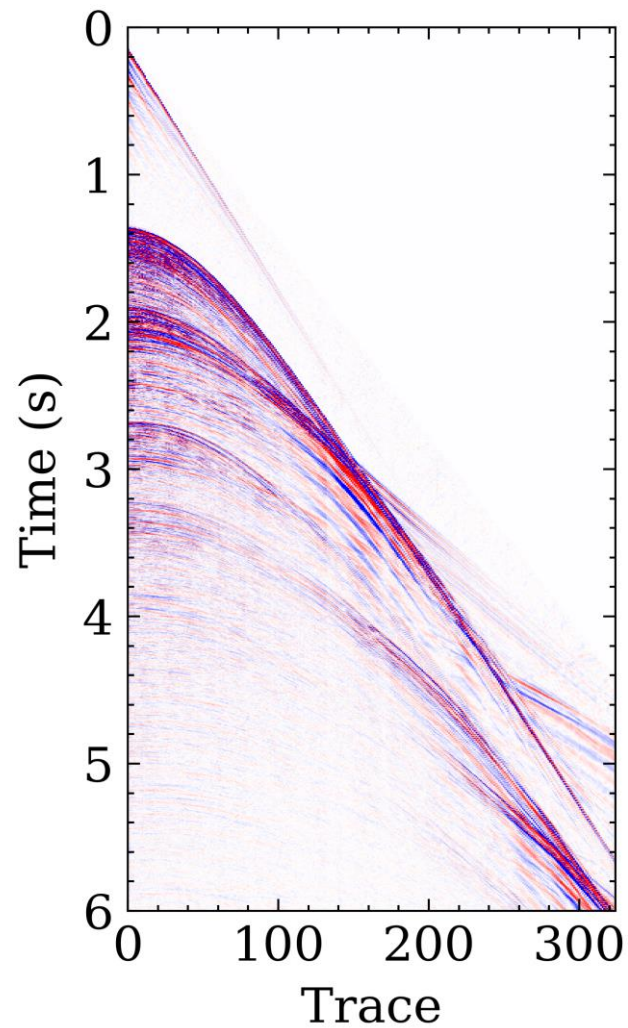
Diffusion FWI framework



using 2D generative diffusion model

Marine Field

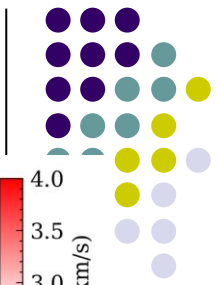
Total 116 shots



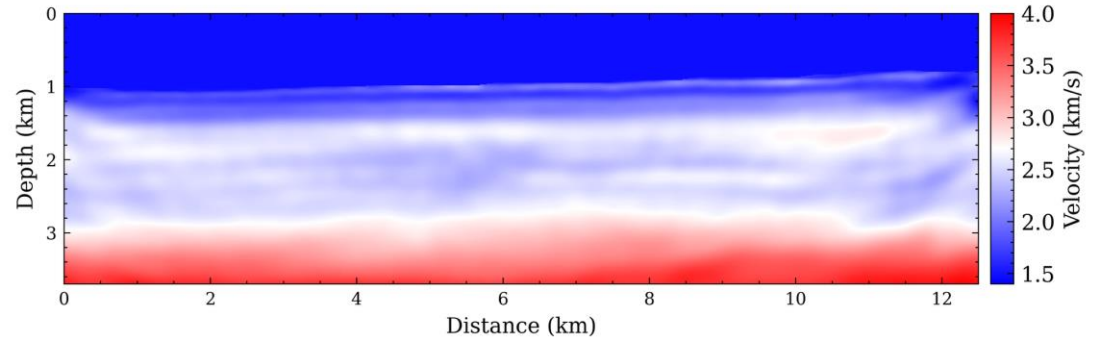
using 2D generative diffusion model



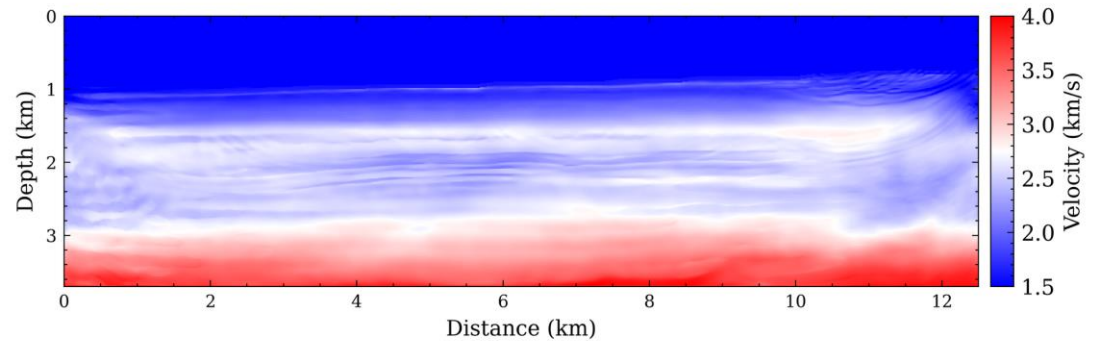
Field Data results



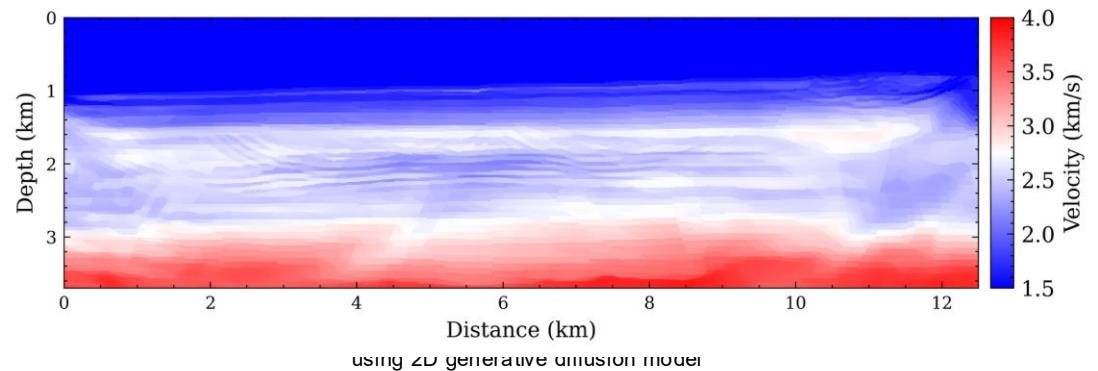
Initial velocity model



Conventional FWI

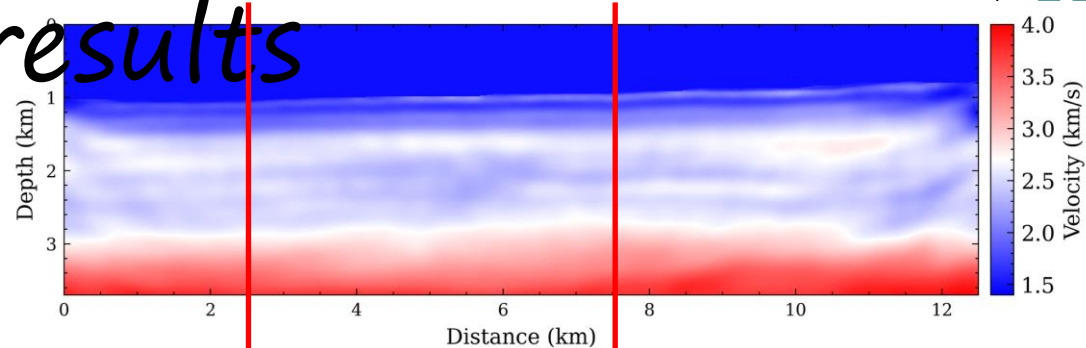


Diffusion FWI by
OpenFWI dataset

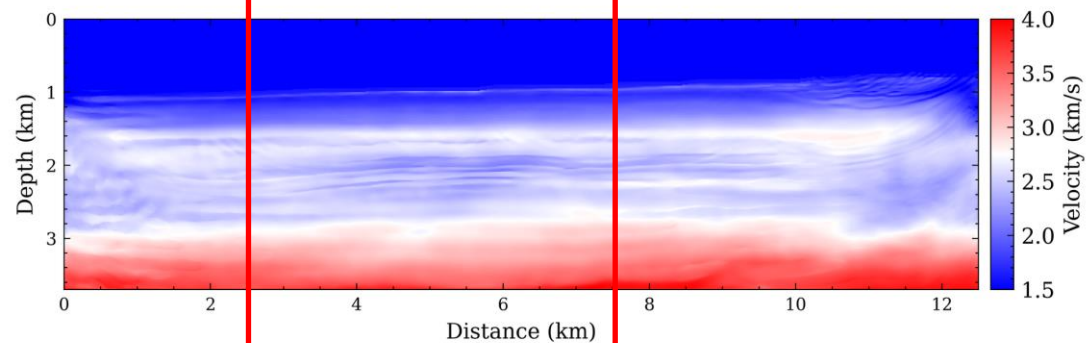


Field Data results

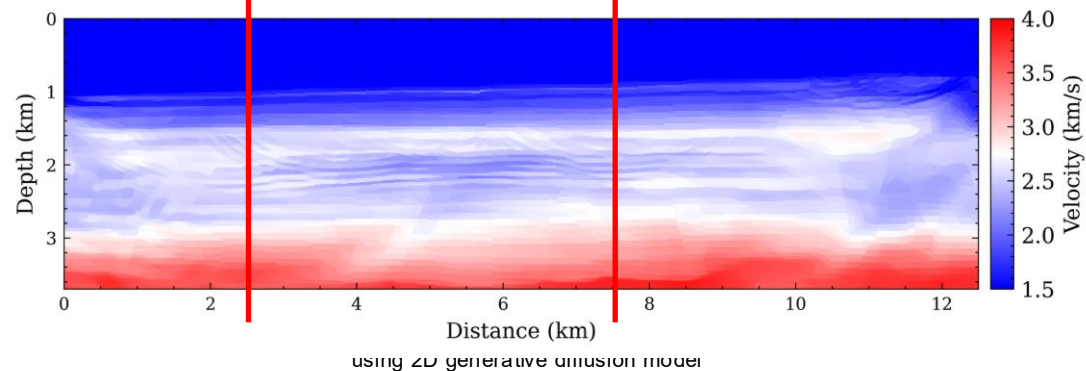
Initial velocity model



Conventional FWI



Diffusion FWI by
OpenFWI dataset

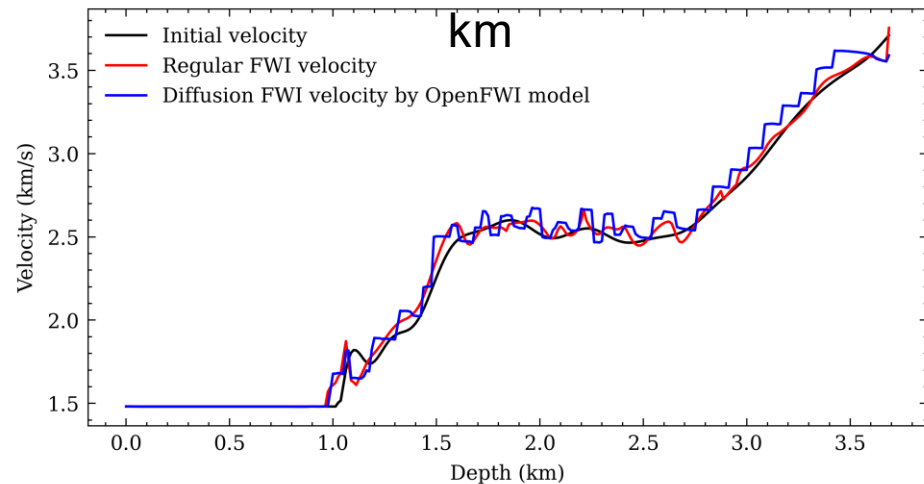


Profile comparisons



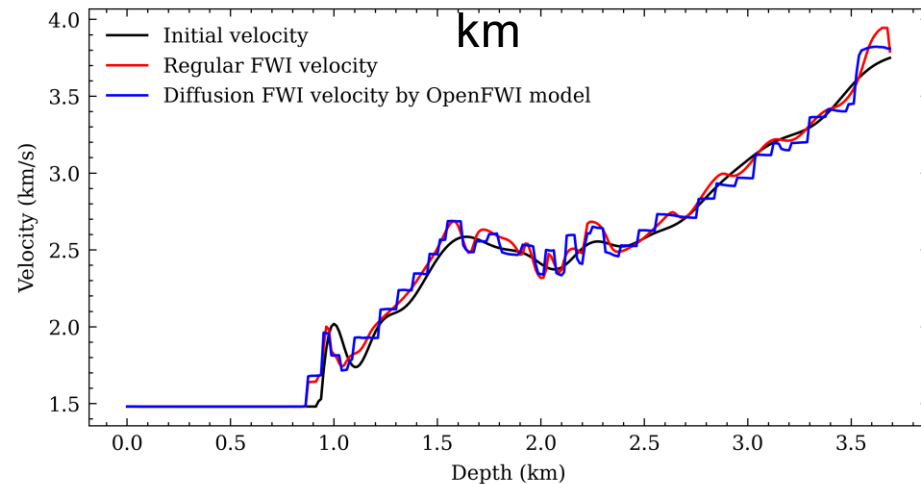
2.5

km

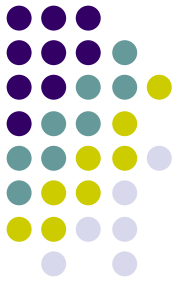


7.5

km

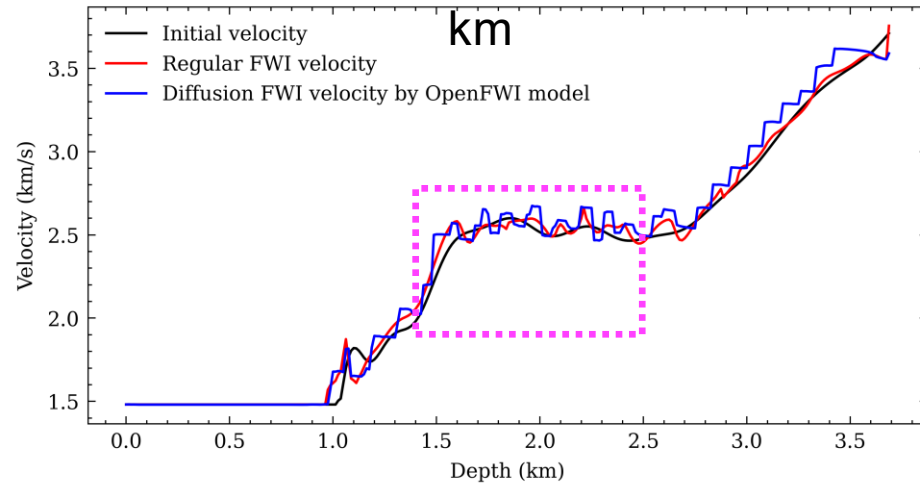


Profile comparisons



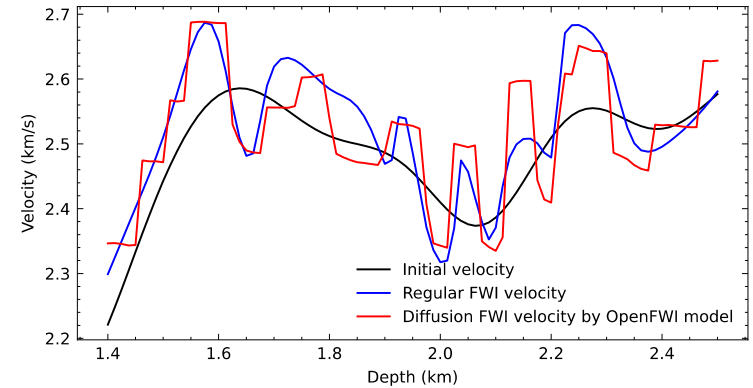
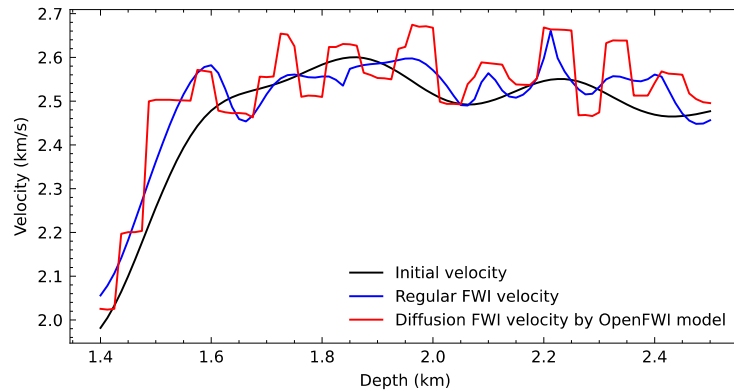
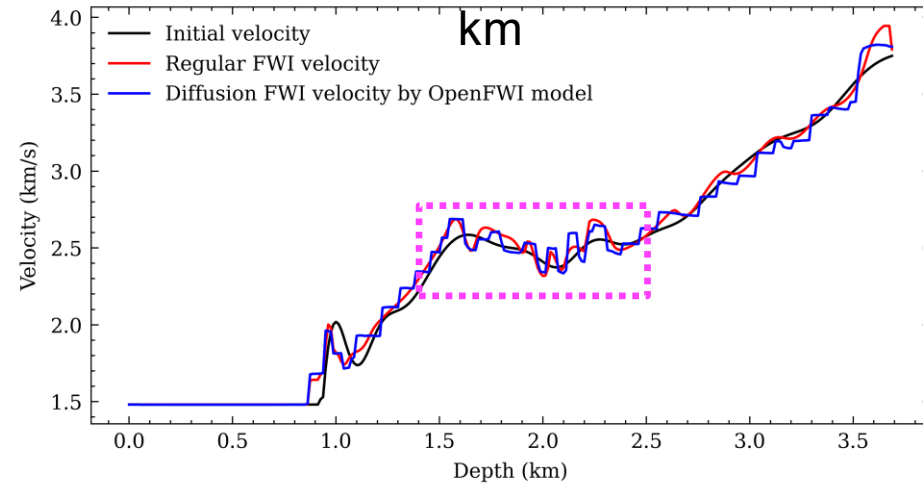
2.5

km

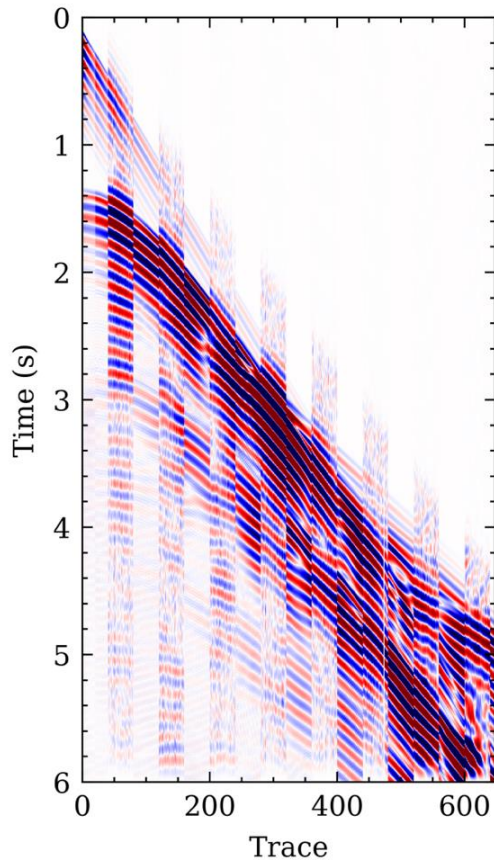
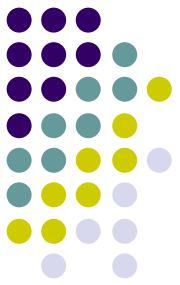


7.5

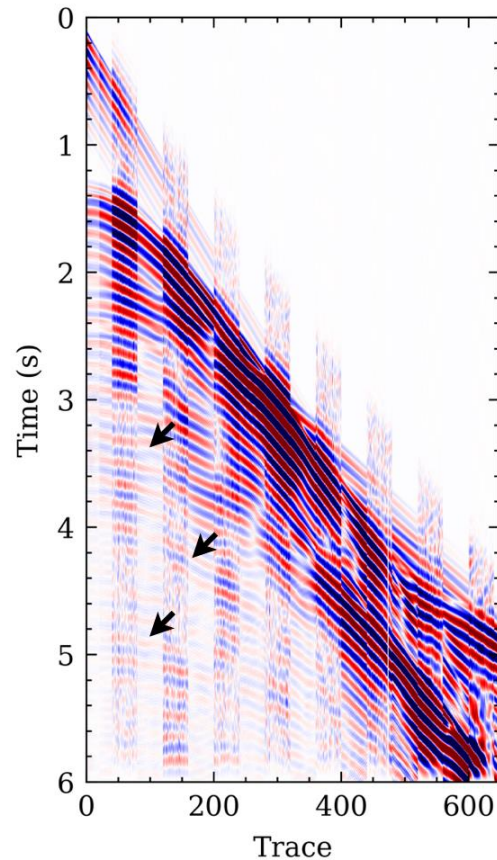
km



The interleaved predicted and observed data

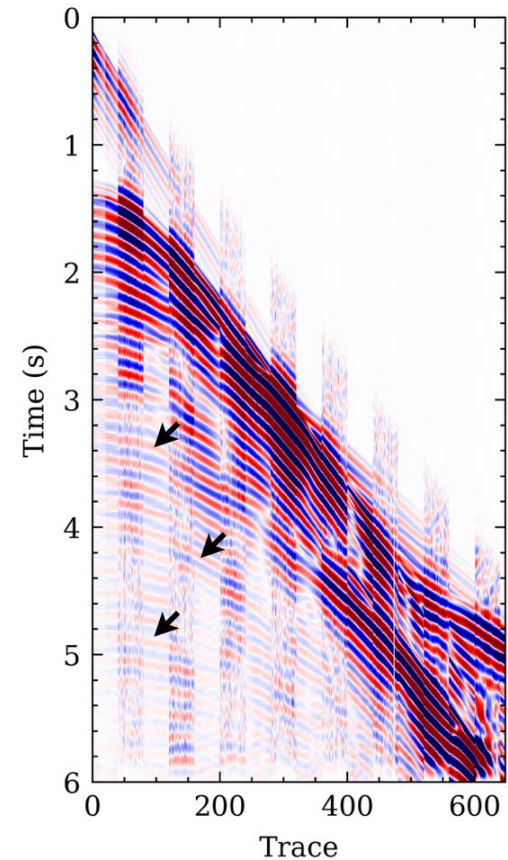


The initial
velocity



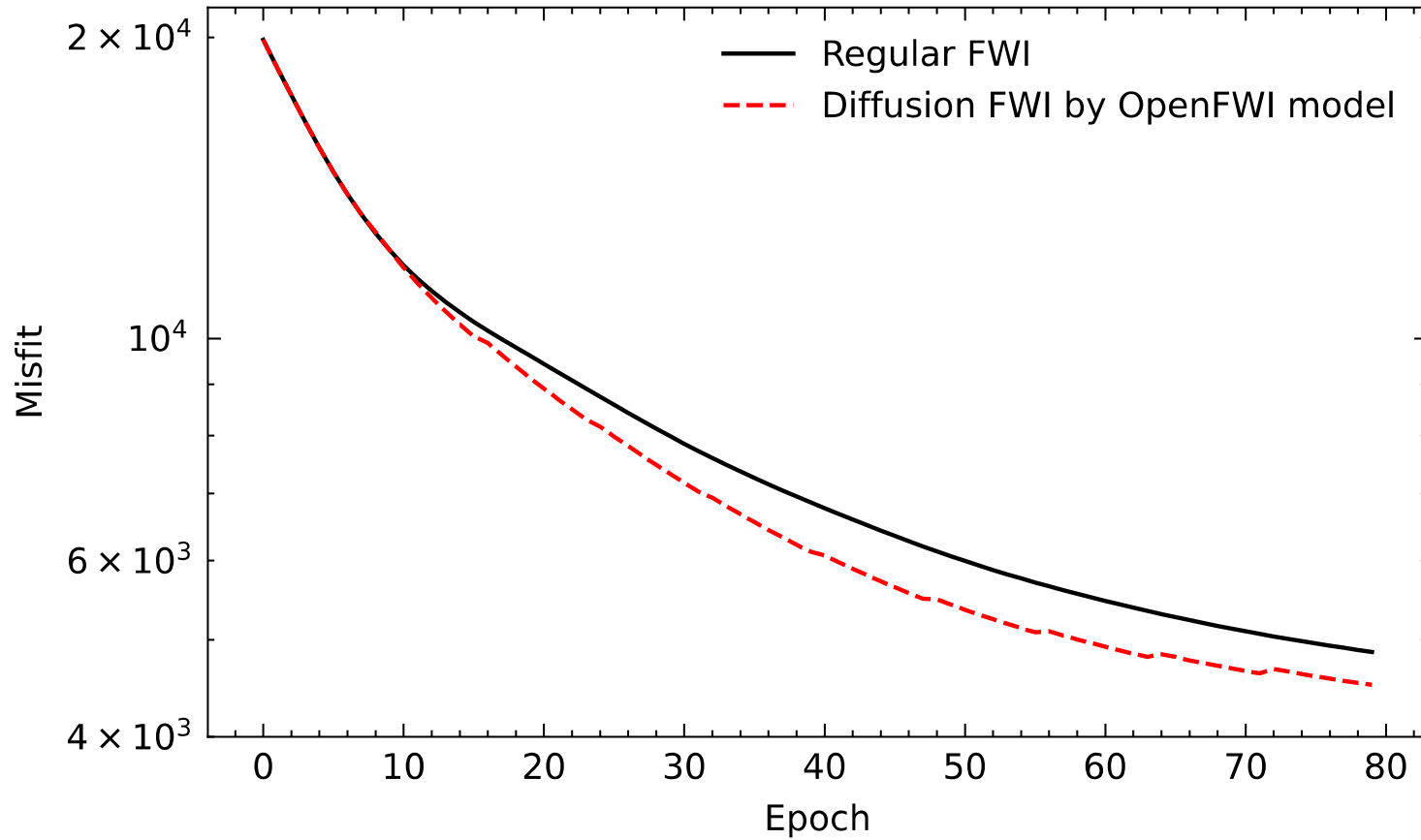
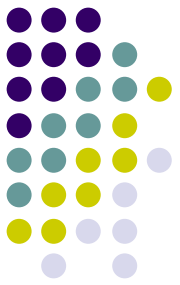
Conventional
FWI

using 2D generative diffusion model

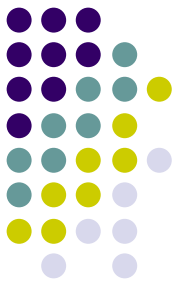


Diffusion FWI

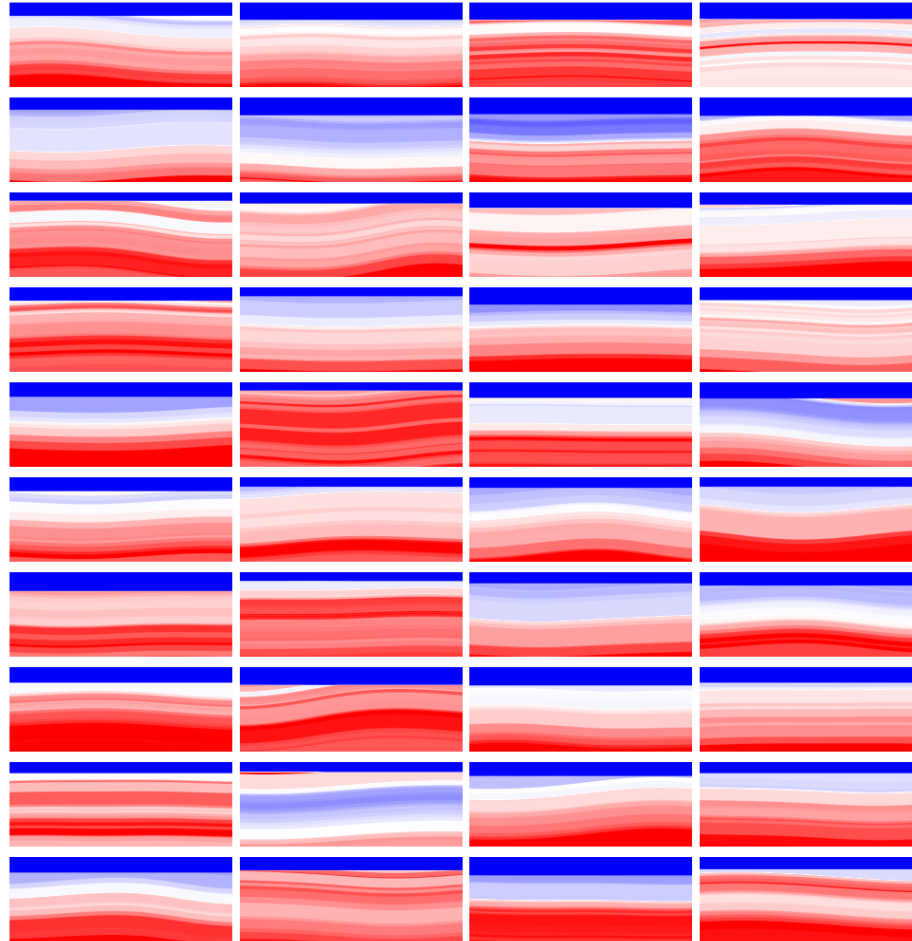
Data misfit



Random model based on well data



Velocity model size:
(296,1000)

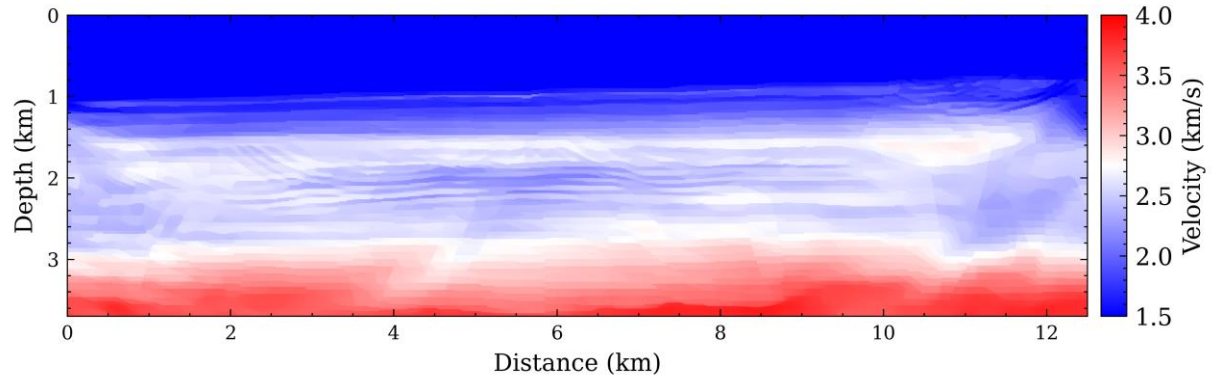


using 2D generative diffusion model

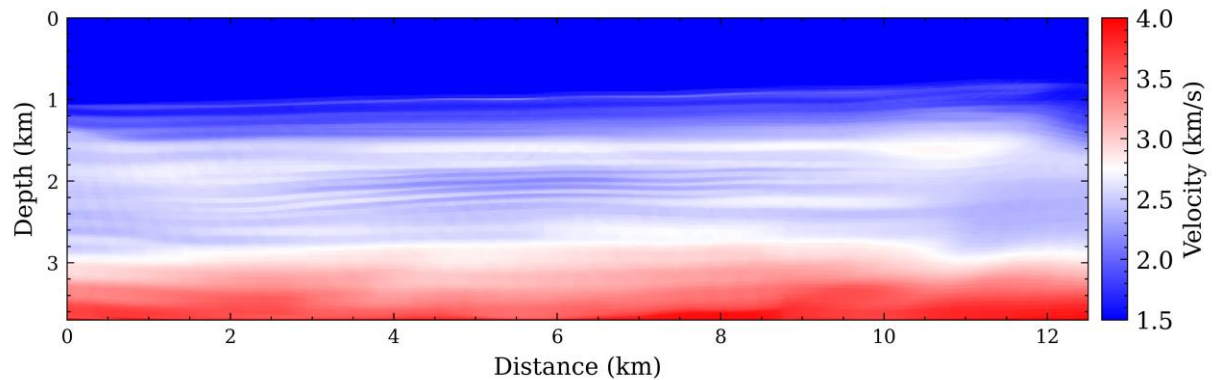
Diffusion FWI results



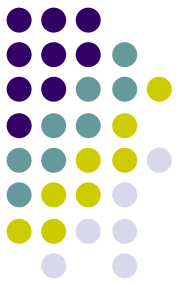
Diffusion FWI by
OpenFWI dataset



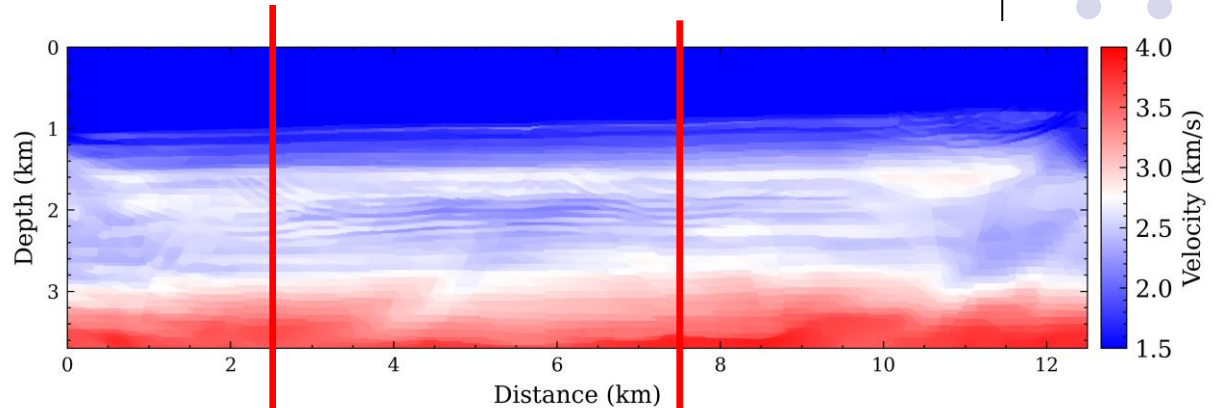
Diffusion FWI by
random model



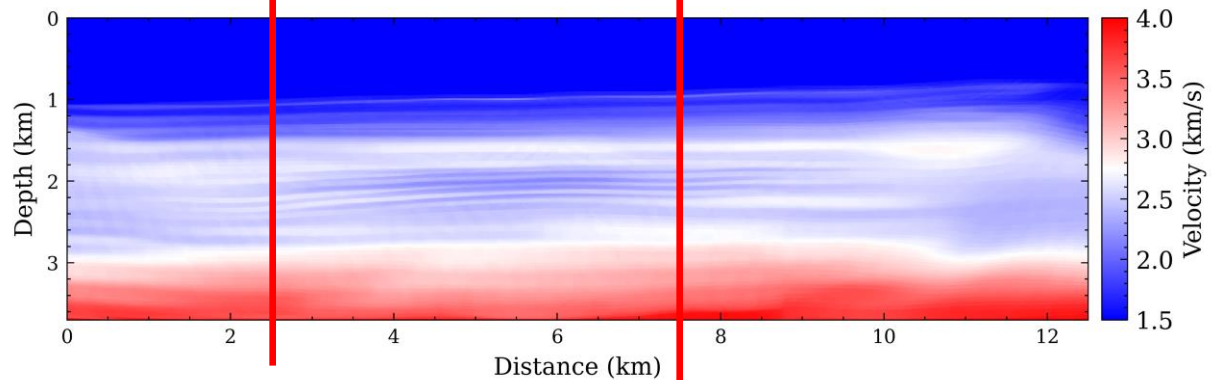
Diffusion FWI results



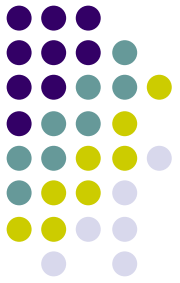
Diffusion FWI by
OpenFWI dataset



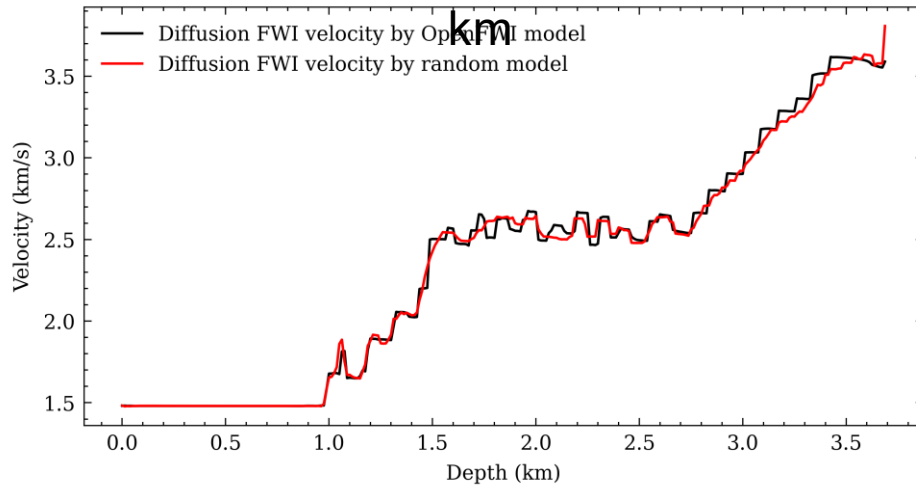
Diffusion FWI by
random model



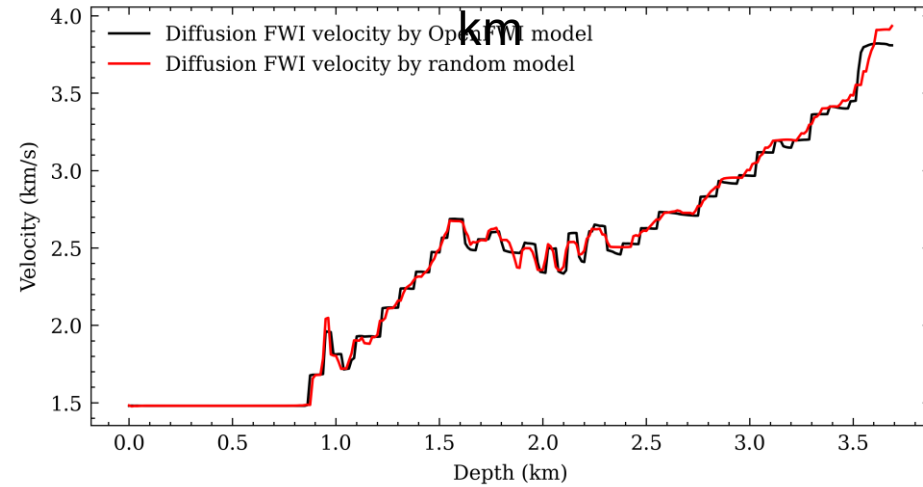
Profile comparisons



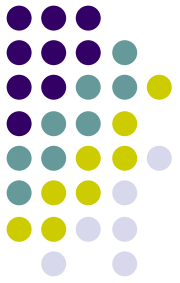
2.5



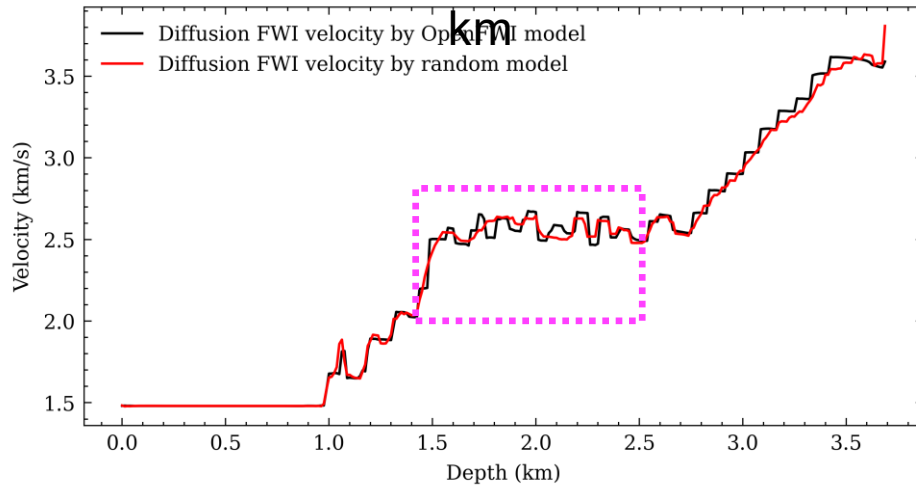
7.5



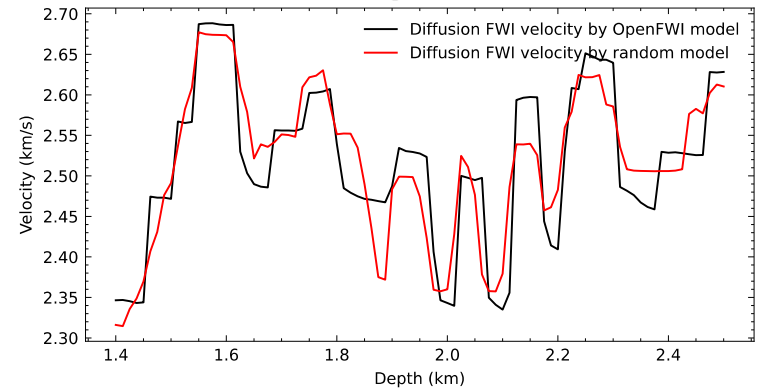
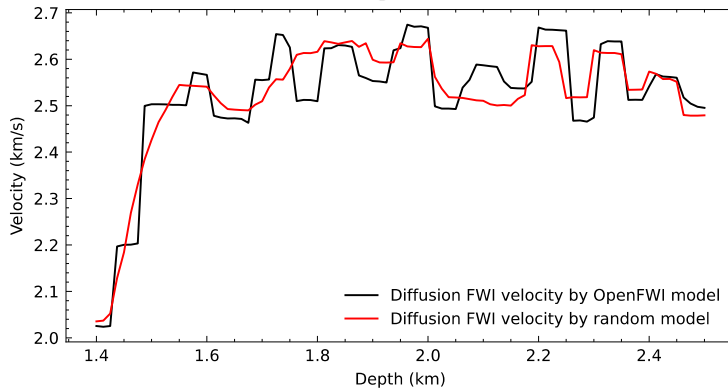
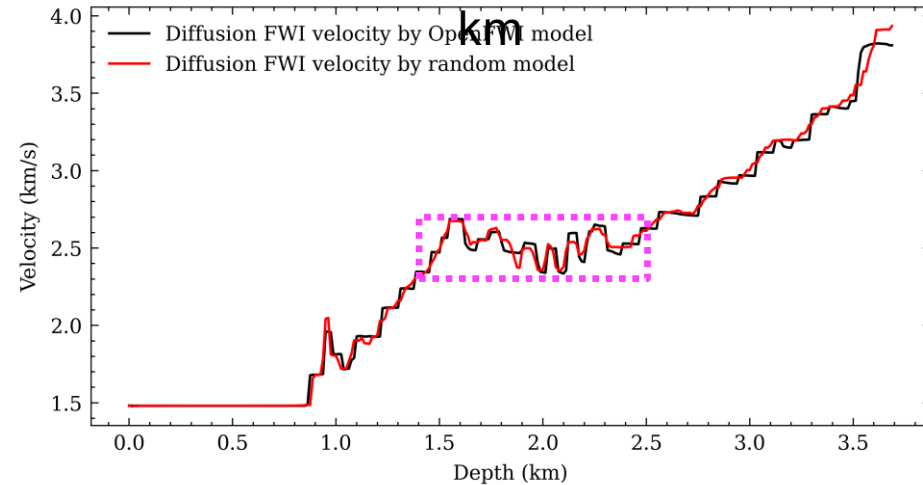
Profile comparisons



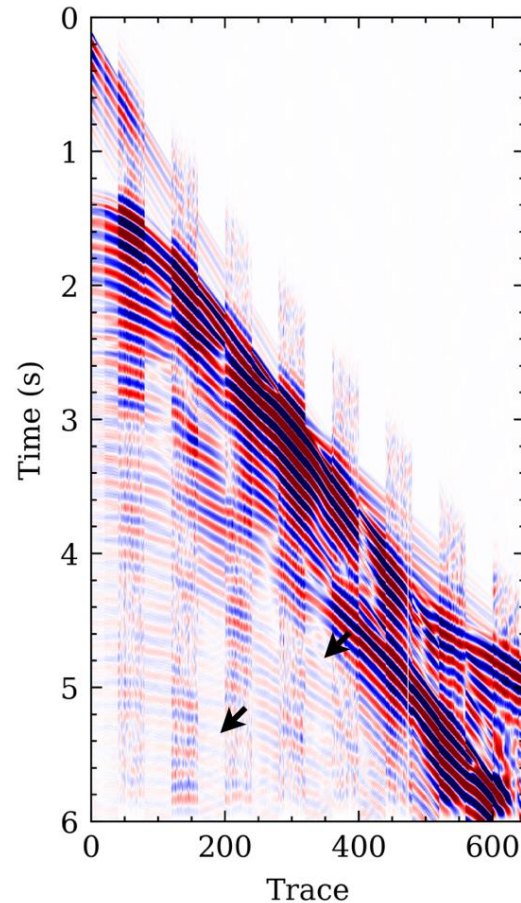
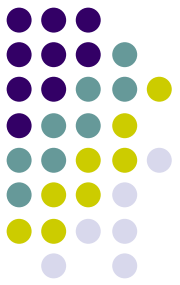
2.5
km



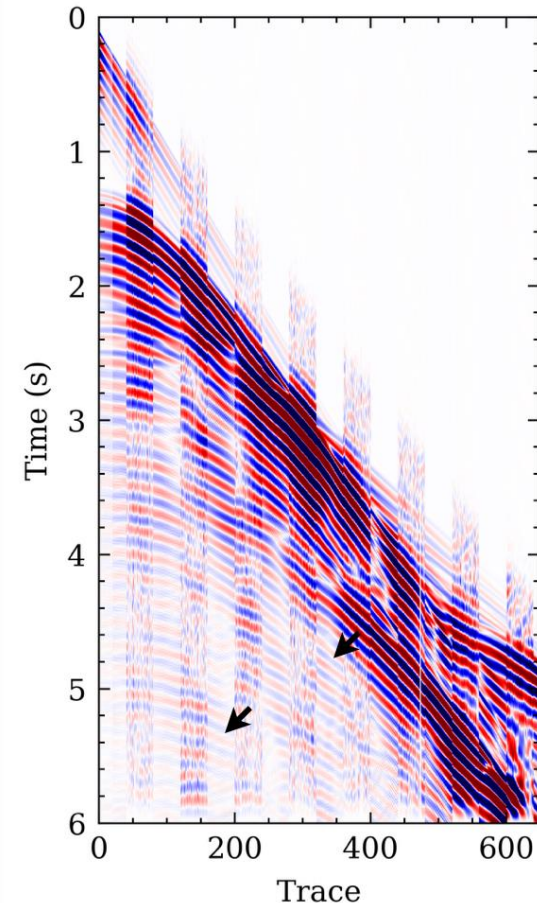
7.5
km



The interleaved predicted and observed data



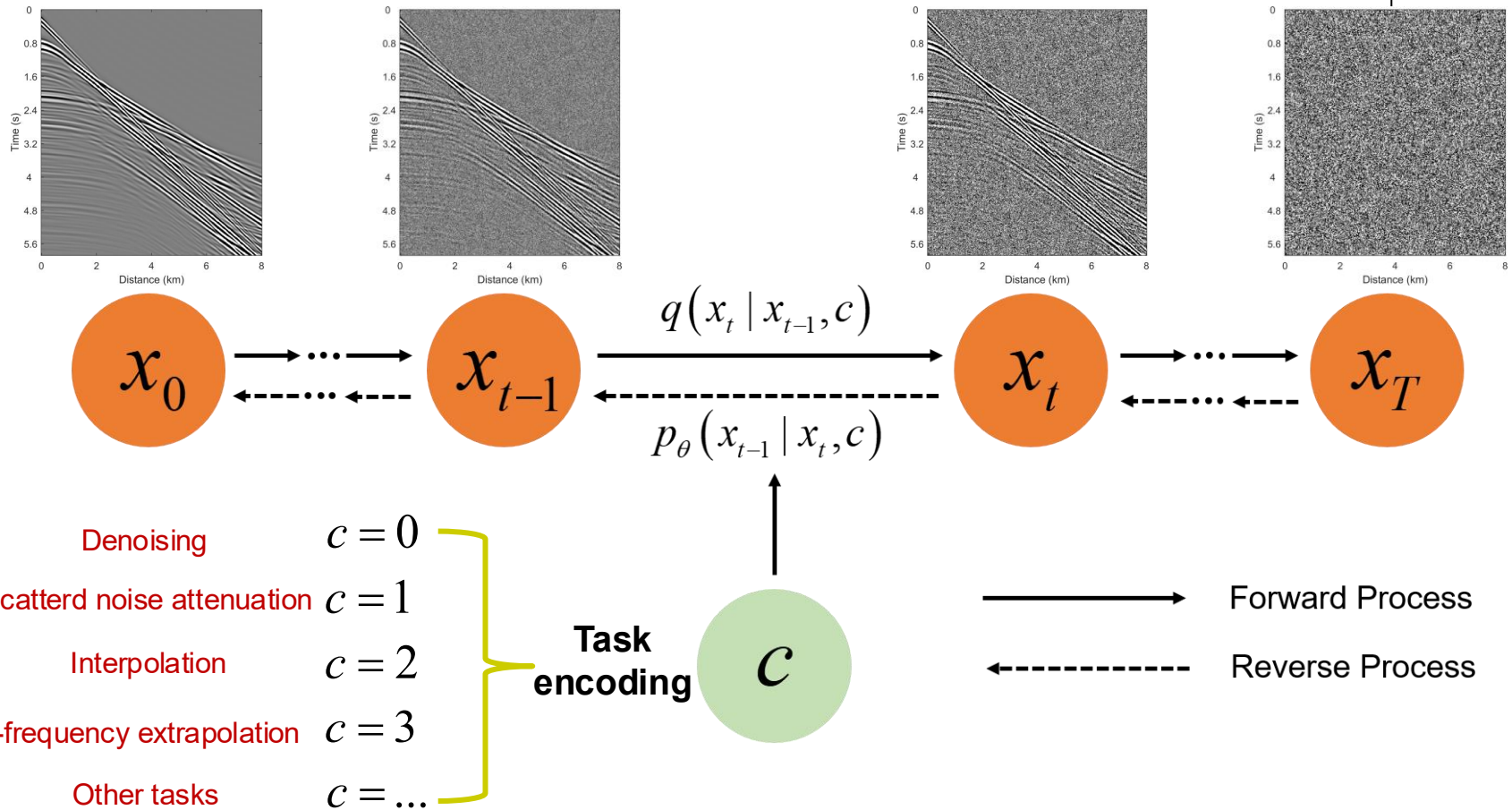
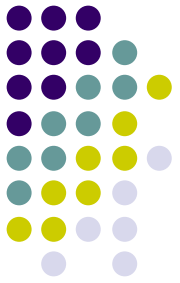
Diffusion FWI by OpenFWI model



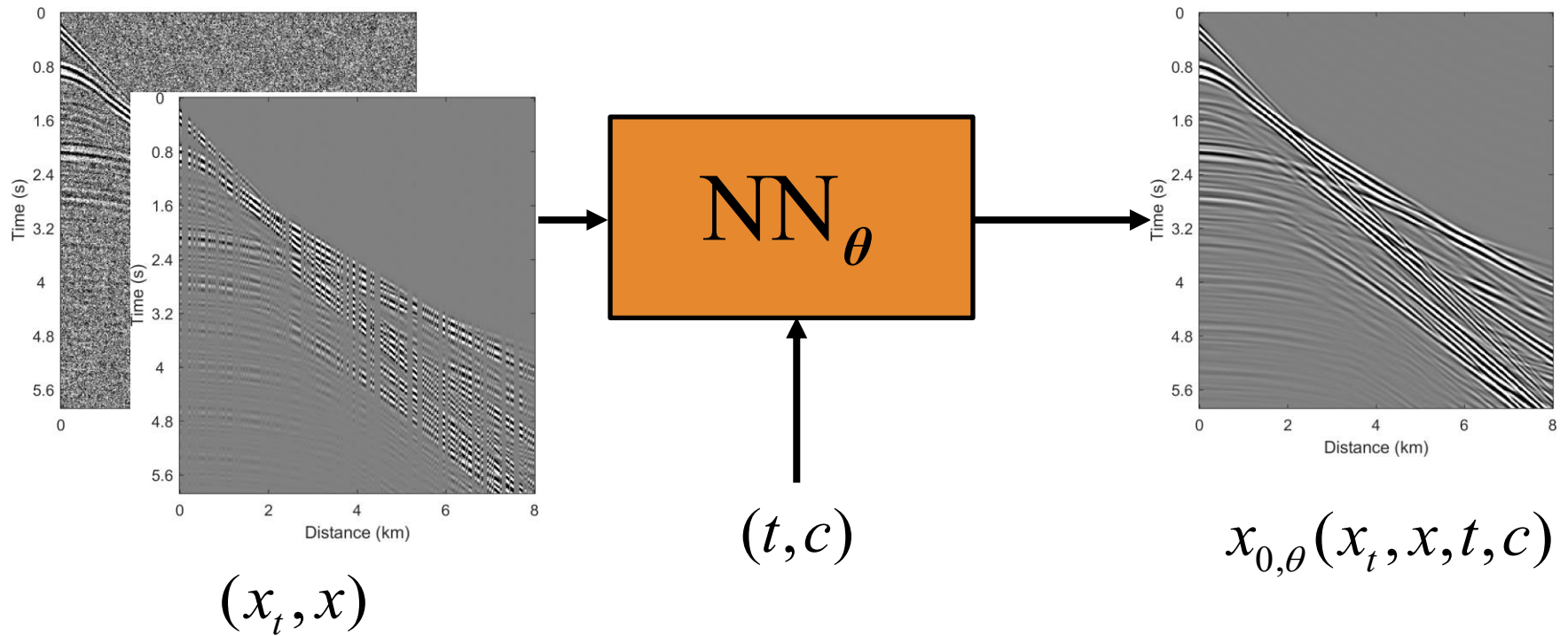
Diffusion FWI by random model

using 2D generative diffusion model

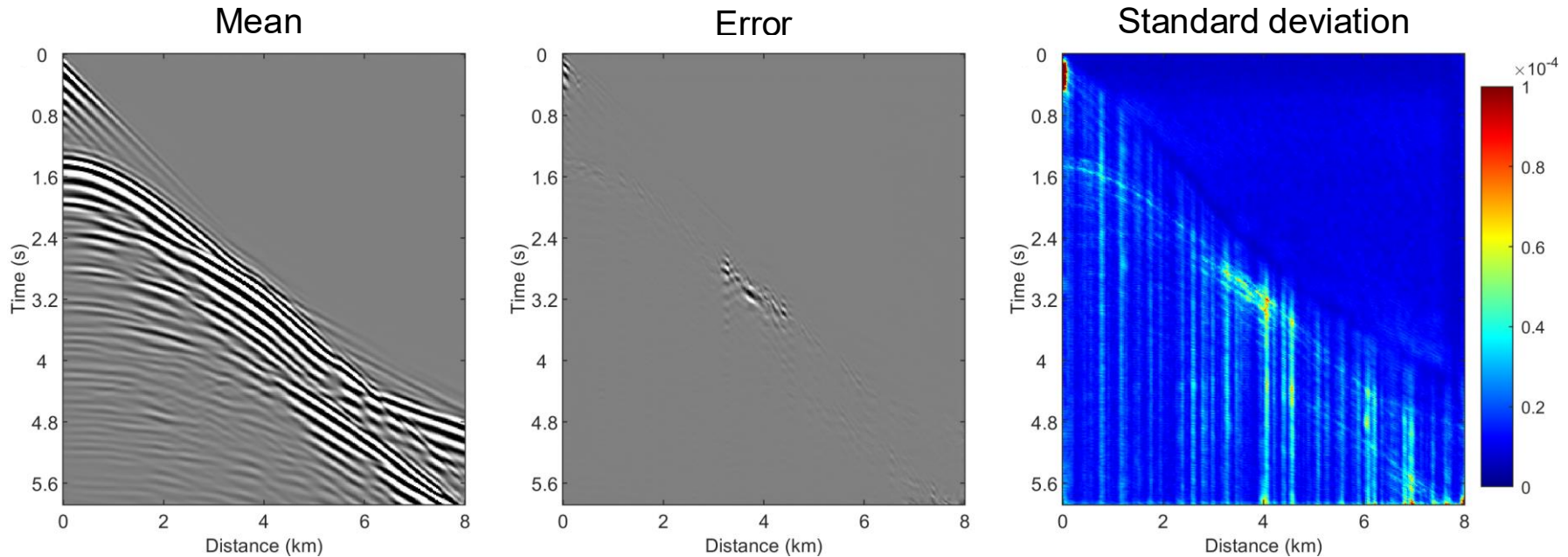
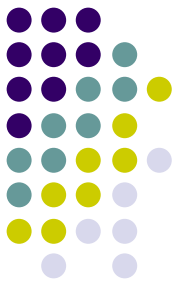
Illustration of our GSFM



Pretraining on synthetic data: Interpolation



Uncertainty quantification using pretrained GSFM





DDPM vs DDIM: Reverse Process Comparison (Sampling Only)

Both methods start from Gaussian noise $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$ and generate samples by reversing the diffusion process.

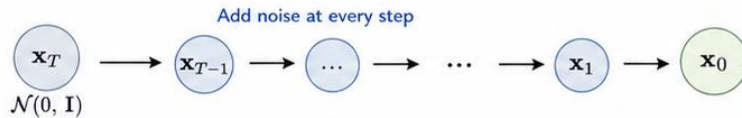
DDPM (Stochastic Reverse Process)

Markovian (depends only on $\mathbf{x}_t$)

Reverse update ($t = T, T-1, \dots, 1$):

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0(\mathbf{x}_t, t) + \sqrt{1 - \bar{\alpha}_{t-1}} \epsilon_\theta(\mathbf{x}_t, t) + \sigma_t \mathbf{z}$$

$\mathbf{z} \sim \mathcal{N}(0, \mathbf{I})$ (random noise), $\sigma_t > 0$



Properties:

- Stochastic (different noise each run $\rightarrow$ different samples)
- Must use all T steps (Markov chain)

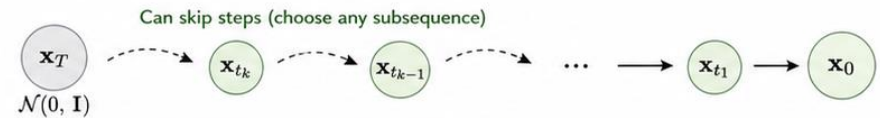
DDIM (Deterministic / Non-Markovian Reverse Process)

Non-Markovian (depends on $\mathbf{x}_t$ and $\hat{\mathbf{x}}_0$)

Reverse update ($t = T, T-1, \dots, 1$):

$$\mathbf{x}_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{\mathbf{x}}_0(\mathbf{x}_t, t) + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(\mathbf{x}_t, t) + \sigma_t \epsilon$$

ϵ can be noise or zero, $\sigma_t \in [0, \sqrt{1 - \bar{\alpha}_{t-1}}]$



Properties:

- Non-Markovian (uses prediction of $\mathbf{x}_0$)
- Can set $\sigma_t = 0$ for deterministic sampling (no randomness)
- Can use fewer steps (skipping possible)

Aspect	DDPM	DDIM
Process type	Markovian (stochastic)	Non-Markovian (deterministic or stochastic)
Update rule	Adds random noise at every step ($\sigma_t > 0$)	Generalized update with $\sigma_t \in [0, \sqrt{1 - \bar{\alpha}_{t-1}}]$
Sampling steps	Must use all T steps	Can use any subset of steps ($T > t_K > \dots > t_1 > 0$)
Randomness	Always stochastic	Can be deterministic ($\sigma_t = 0$) or stochastic ($\sigma_t > 0$)
Speed	Slower (many steps needed)	Faster (fewer steps possible)
Sample quality	High (with enough steps)	Comparable quality with fewer steps



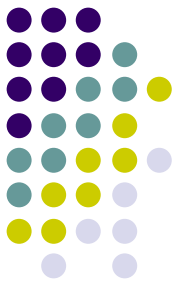
Key Takeaway

DDIM generalizes DDPM.

DDPM is recovered when $\sigma_t = \sqrt{\beta_t}$, while $\sigma_t = 0$ gives deterministic sampling.

DDIM enables fast sampling with fewer steps while maintaining similar sample quality.

Conclusions



- The generative power has only begun, stay tuned
- Sampling from complex distributions in inverse problems
- We can incorporate our expectations, the data and physics
- Applications include tomography, prediction, inversion,.....